

Streaming

MLOPs II

Jaime A Riascos-Salas

2026

Contenidos - MLOps2

Tema	Sesión
Modos de Flujo de Datos y Patrones Avanzados de APIs REST	1 (27/08/26)
Grafos aplicados a bases de datos (Neo4j) y APIs - GraphQL en MLOps - Flexibilidad en la Consulta de Datos	2 (03/09/26)
gRPC para Microservicios de ML de Alta Performance	3 (10/09/26)
Procesamiento de Datos en Streaming para Machine Learning	4 (17/09/26)
Seguridad en APIs	5 (24/09/26)
Procesamiento en la Nube y Data Lakes para MLOps	6 (01/10/26)
Aprendizaje Federado - Privacidad y Descentralización en ML	7 (08/10/26)
Sesión de Práctica Integrada / Taller de Proyecto Final	8 (15/10/26)

Contenidos - MLOps2

Tema	Sesión
Modos de Flujo de Datos y Patrones Avanzados de APIs REST	1 (27/08/26)
Grafos aplicados a bases de datos (Neo4j) y APIs - GraphQL en MLOps - Flexibilidad en la Consulta de Datos	2 (03/09/26)
gRPC para Microservicios de ML de Alta Performance	3 (10/09/26)
Procesamiento de Datos en Streaming para Machine Learning	4 (17/09/26)
Aprendizaje Federado - Privacidad y Descentralización en ML	5 (24/09/26)
Seguridad en APIs	6 (01/10/26)
Procesamiento en la Nube y Data Lakes para MLOps	7 (08/10/26)
Sesión de Práctica Integrada / Taller de Proyecto Final	8 (15/10/26)

Tipos de datos

Bounded data: los datos acotados son finitos y tienen un principio y un final discretos. Está asociado con el procesamiento por lotes.

Unbounded data: los datos ilimitados son infinitos y no tienen un principio ni un final discretos. Está asociado con el procesamiento de flujos.



Tipos de procesamiento

Hay tres formas en que las organizaciones trabajan con datos ilimitados (Unbounded).

1. Procesamiento por lotes de datos almacenados: los datos ilimitados se almacenan y procesan a intervalos especificados.
2. Procesamiento de eventos: cada evento en el flujo ilimitado se maneja por separado con conexiones entre eventos que se almacenan en almacenamiento persistente.
3. Procesamiento de flujos: procesamiento continuo de flujos de datos de gran volumen. Los datos se procesan en la memoria antes de almacenarlos

Tipos de procesamiento

	Batch Processing	Stream Processing
Frequency	Infrequent jobs that produce results once the job has finished running.	Continuously running jobs that produce constant results.
Performance	High latency (minutes to hours).	Low latency (seconds to milliseconds).
Data Sources	Databases, APIs, and static files.	Message queues, event streams, and transactions.
Analysis Type	Complex analysis is possible.	Simpler analysis, including aggregation, filtering, enrichment, proximity analysis, and event detection.
Processing	Process after storing the data.	Process and then maybe store the data.

Stream processing

El procesamiento streaming o de flujos es un término que agrupa la recopilación, integración y análisis de datos Unbounded o sin limite. El procesamiento de flujos ofrece información a las organizaciones de forma continua

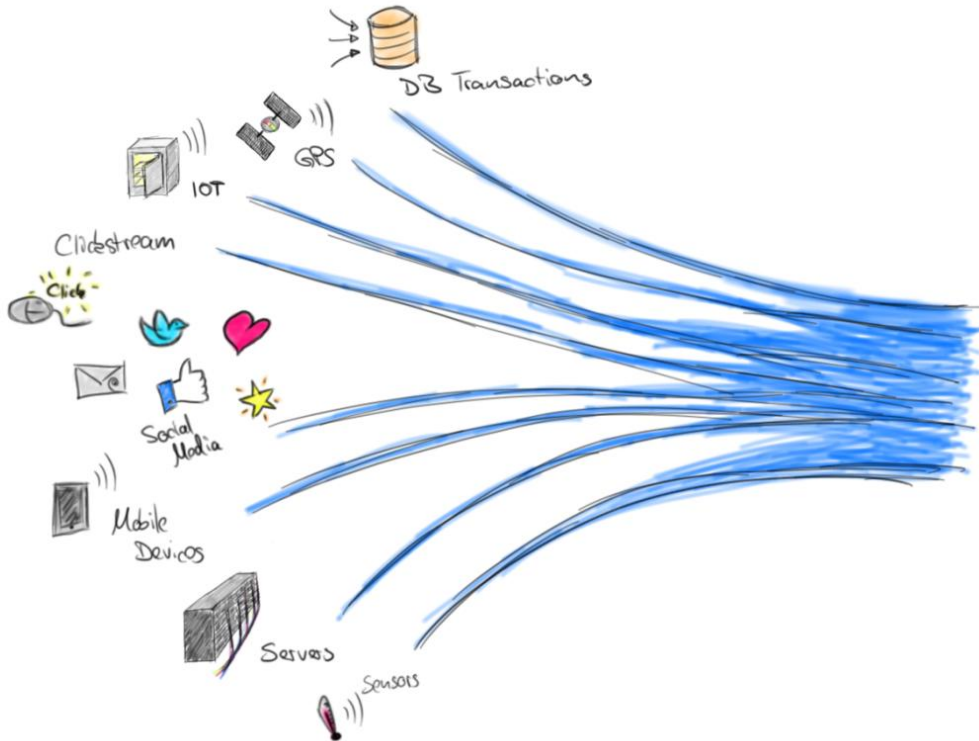
Stream processing

El procesamiento “Streaming” es el acto de incorporar continuamente nuevos datos para calcular un resultado



Stream processing

Los datos de entrada son ilimitados. Una serie de eventos, sin principio ni final predeterminados.

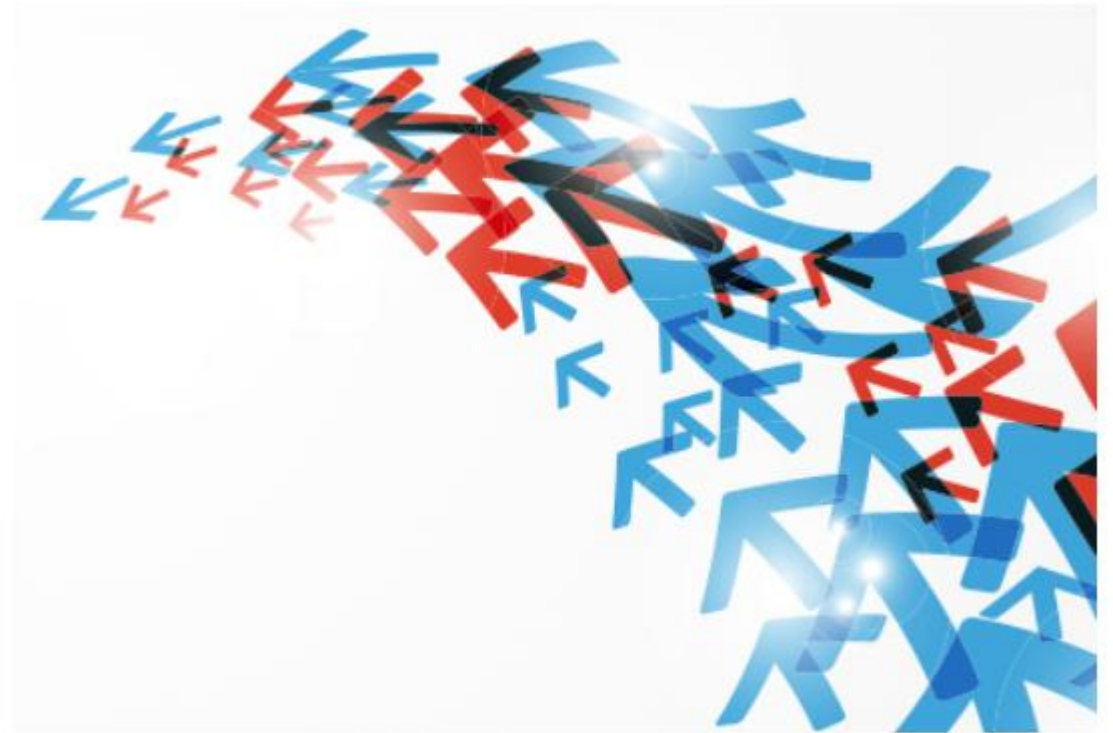


Por ejemplo, transacciones con tarjeta de crédito, clics en un sitio web o lecturas de sensores de dispositivos IoT.

Stream processing

Las aplicaciones de usuario pueden calcular varias consultas en este flujo de eventos.

Por ejemplo, realizar un seguimiento de un recuento acumulado de cada tipo de evento o agregarlos en ventanas horarias.



Stream processing

Sistemas de gestión de bases de datos (DBMS):

Análisis de datos en reposo

Almacena e indexa los datos antes de procesarlos.

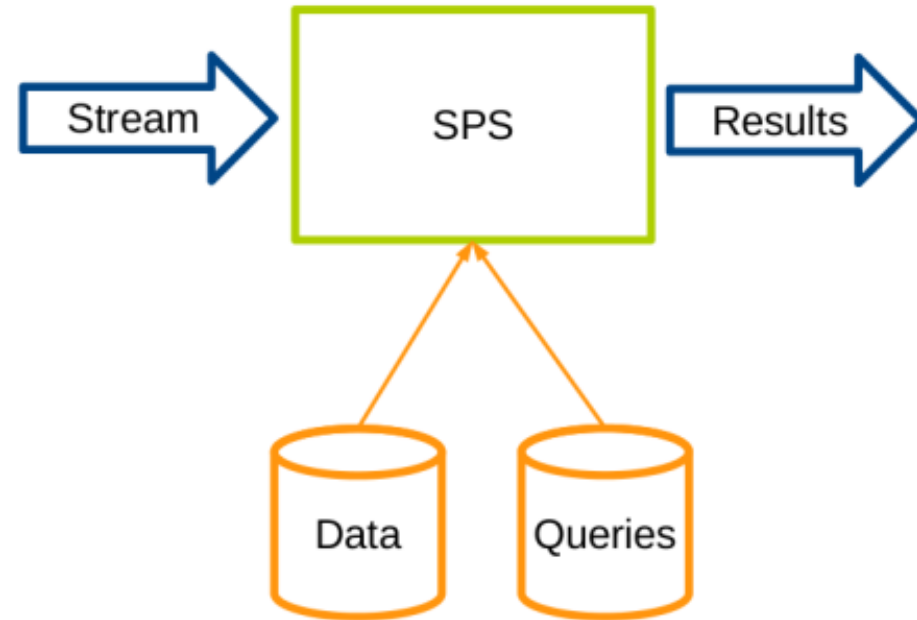
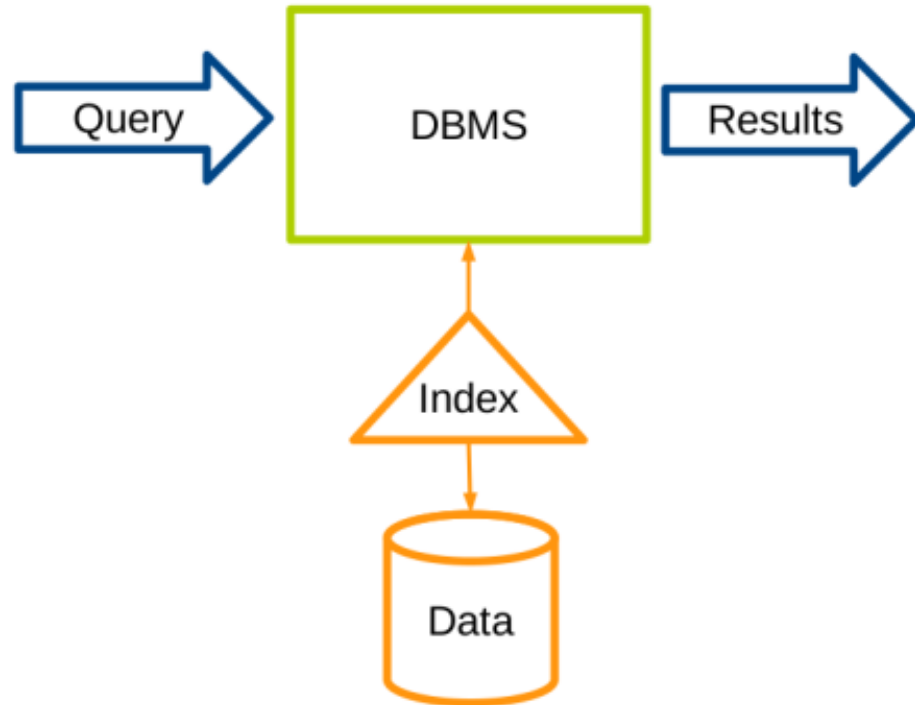
Procese los datos solo cuando los usuarios lo soliciten explícitamente.

Sistemas de procesamiento de flujo (SPS):

Análisis de datos en movimiento

Procesar la información a medida que fluye, sin almacenarla de forma persistente

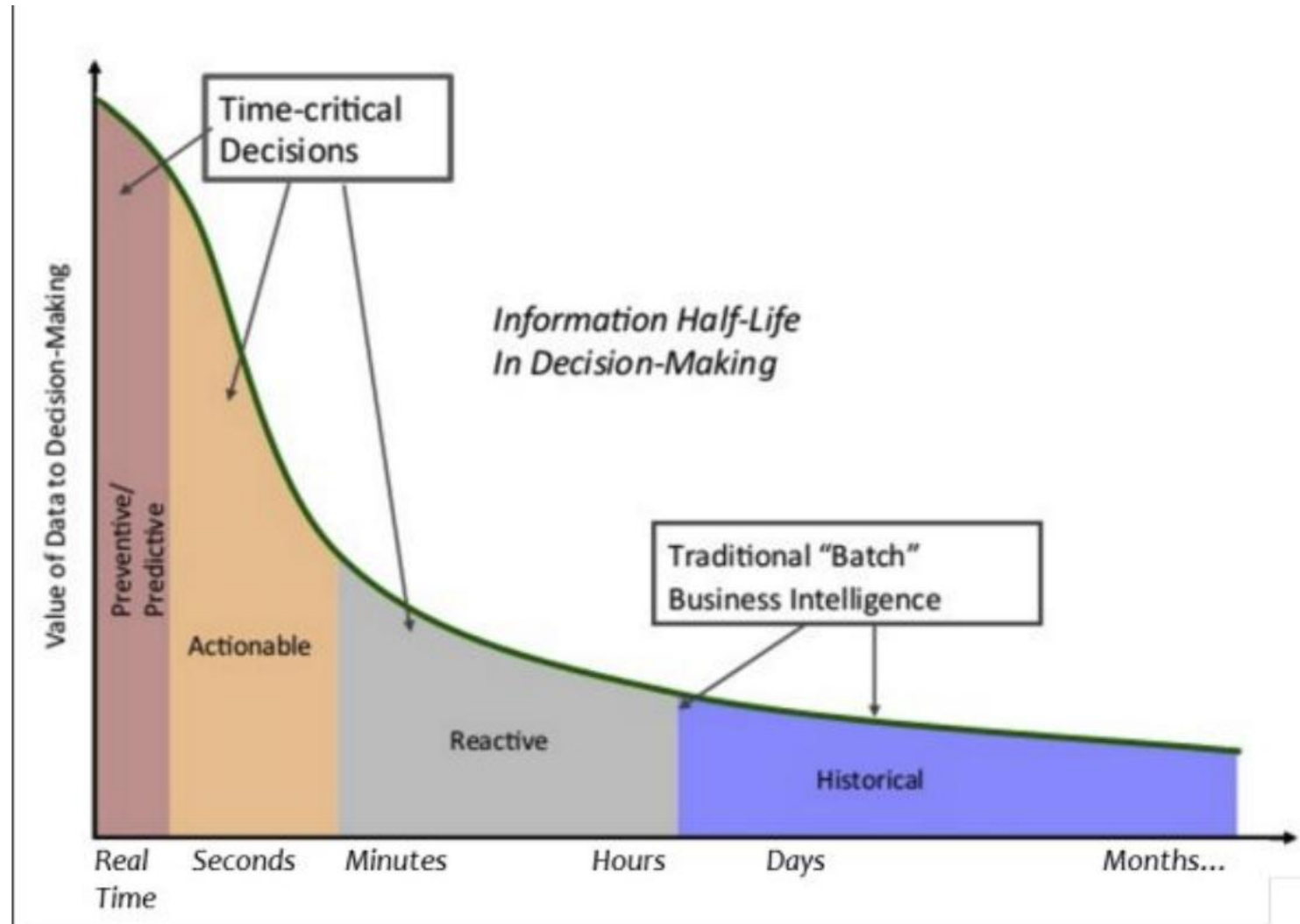
Stream processing



Características de los flujos de datos

- Los registros de datos son de tamaño pequeño.
- Los volúmenes de datos pueden ser extremadamente altos.
- La distribución de datos puede ser inconsistente con períodos tranquilos y ocupados.
- Los datos pueden llegar fuera de secuencia en comparación con el momento en que ocurrió el evento.

Disminución del valor de los datos



Source: Amazon Web Services

Fuente de datos

Business Applications

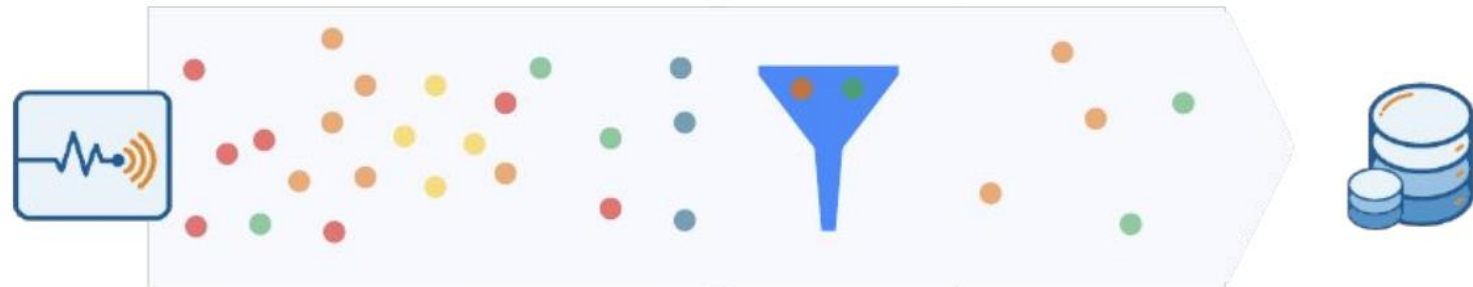
- Customer orders
- Airline reservations
- Insurance claims
- Bank transactions
- Telco call detail records

Digital Information

- Clickstreams
- Social computing
- Customer call logs
- News, weather feeds
- IT, network logs
- Market data
- Email

Internet of Things

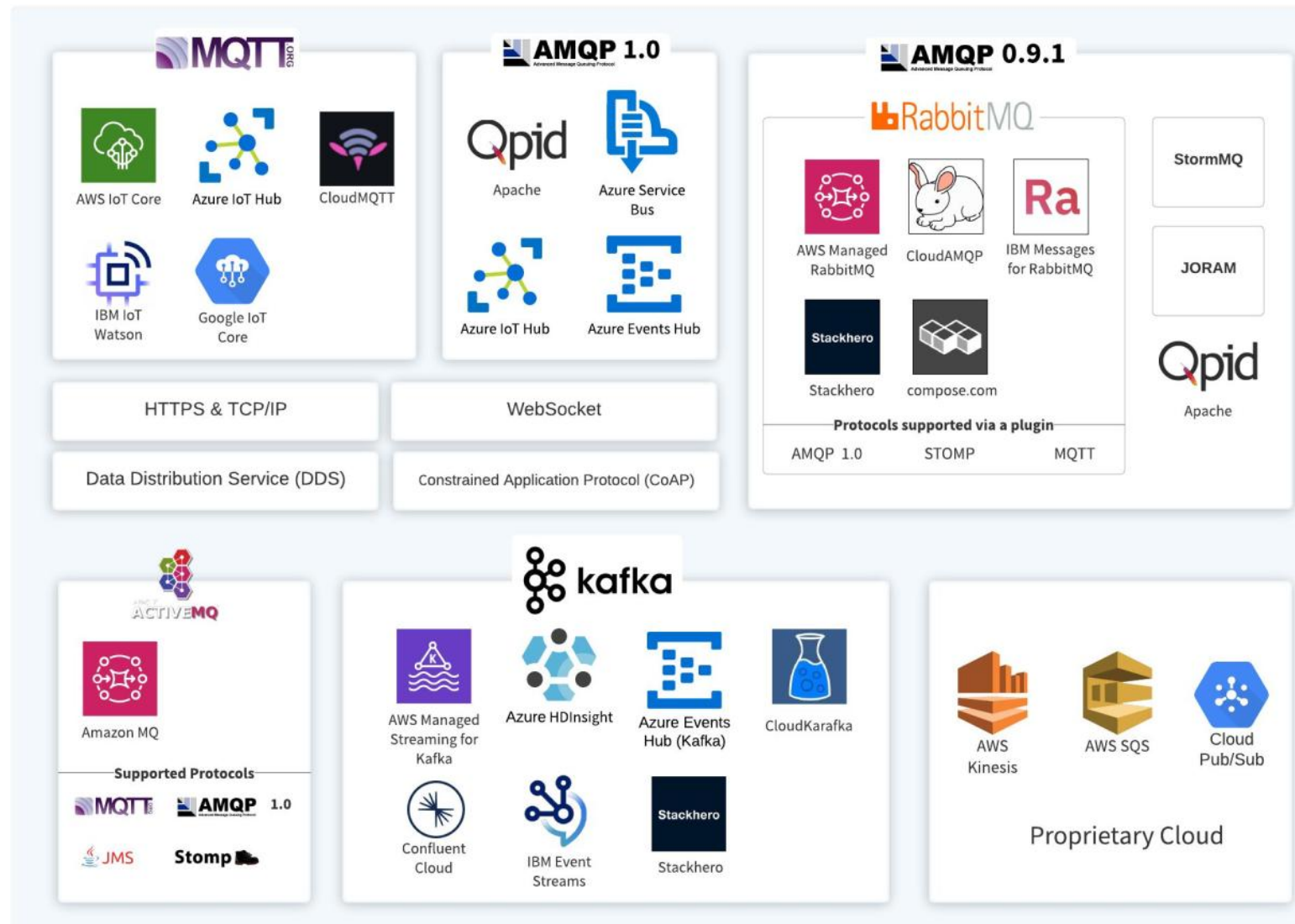
- Radio-frequency Identification
- Telemetry SCADA
- Geolcoation
- Machine logs



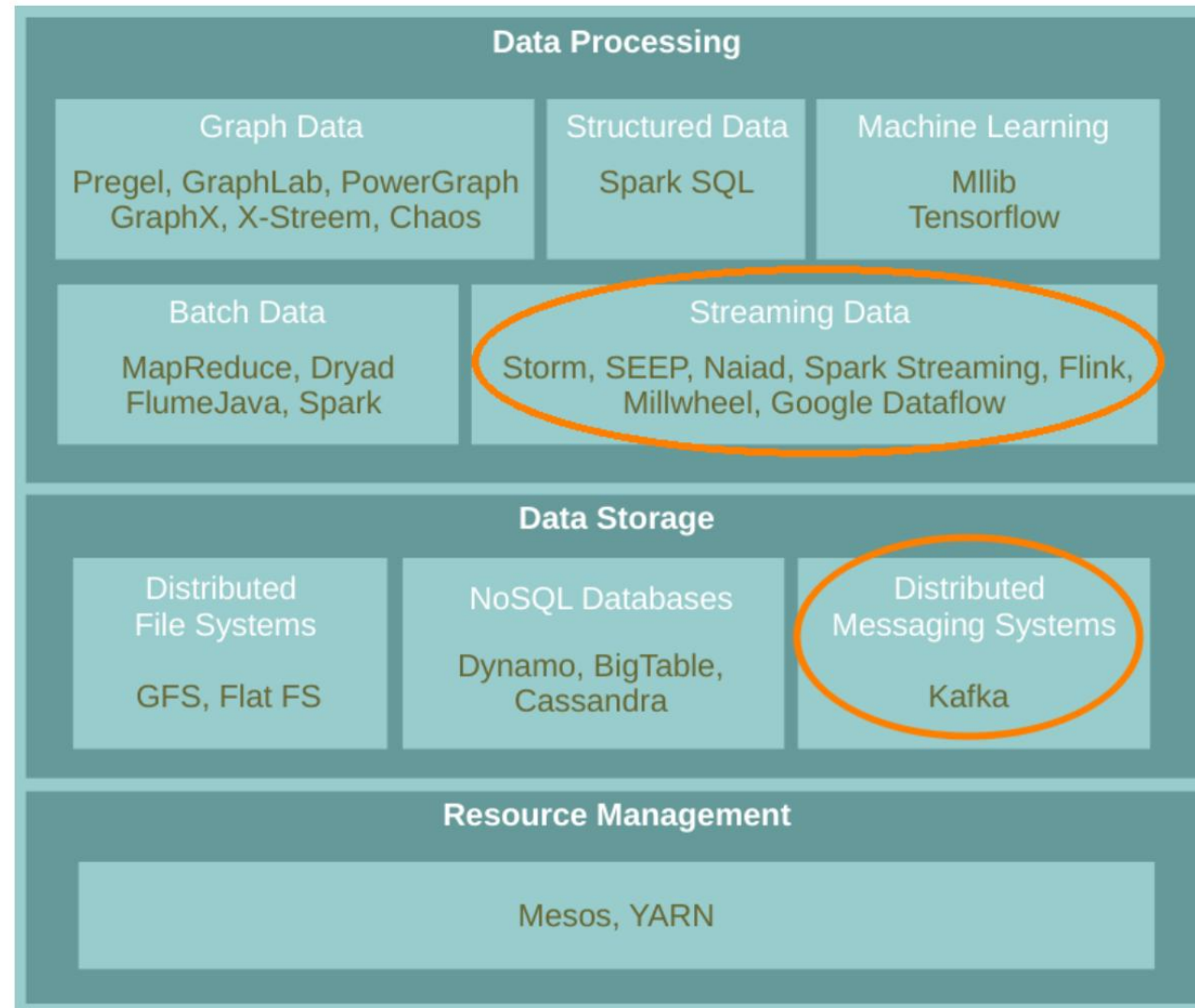
Industrias que se benefician del streaming

- Utilidades
- Energía
- Telecomunicaciones
- Transporte
- Comercial
- Gobierno
- Atención sanitaria

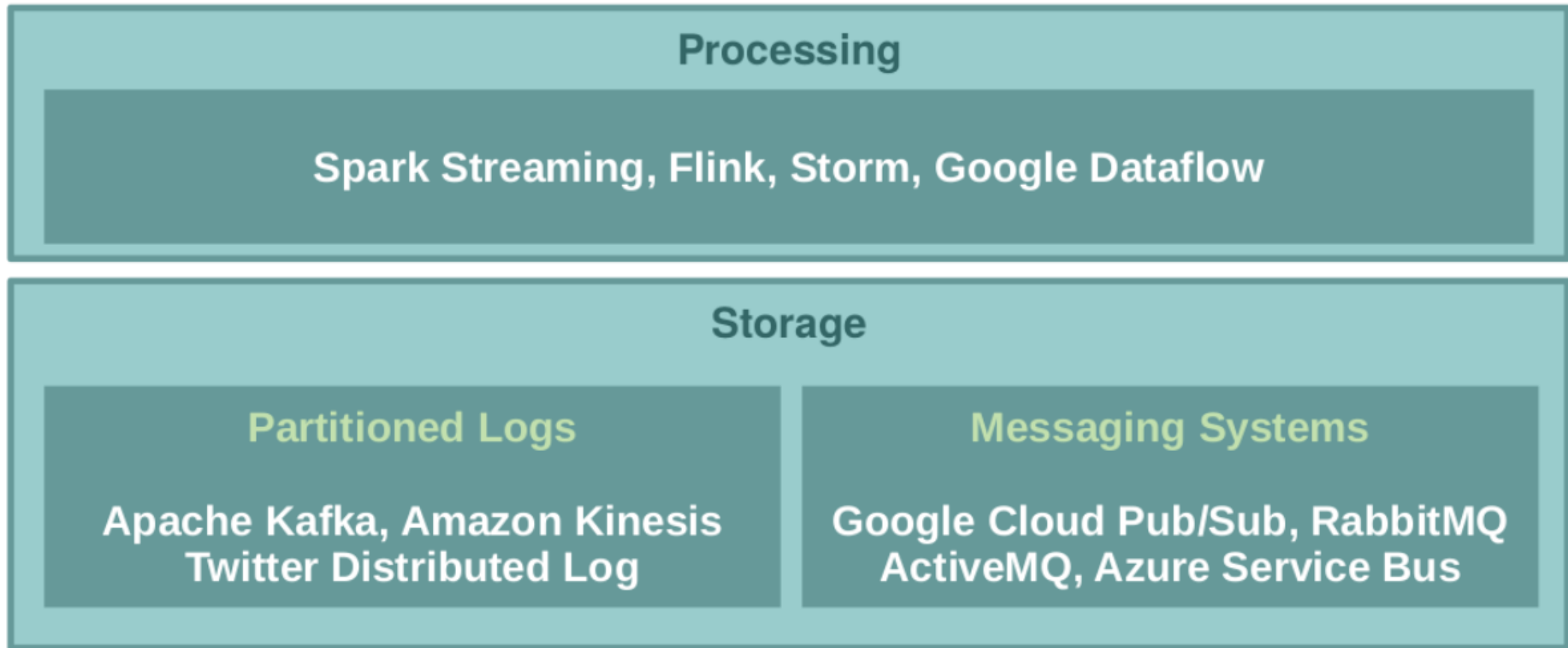
Streaming - Tecnologías



¿Dónde ubicar Streaming?



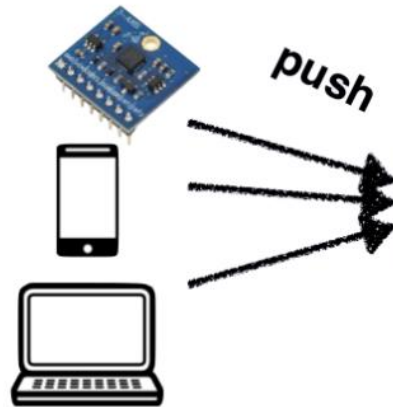
Stream processing



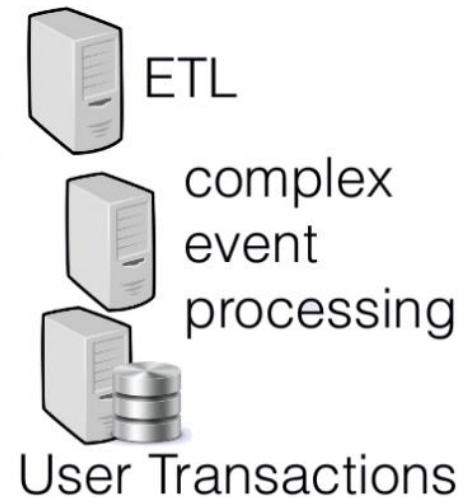
Data Stream Storage

Necesitamos difundir flujos de eventos de varios productores a varios consumidores.

Data Producers



Data Consumers



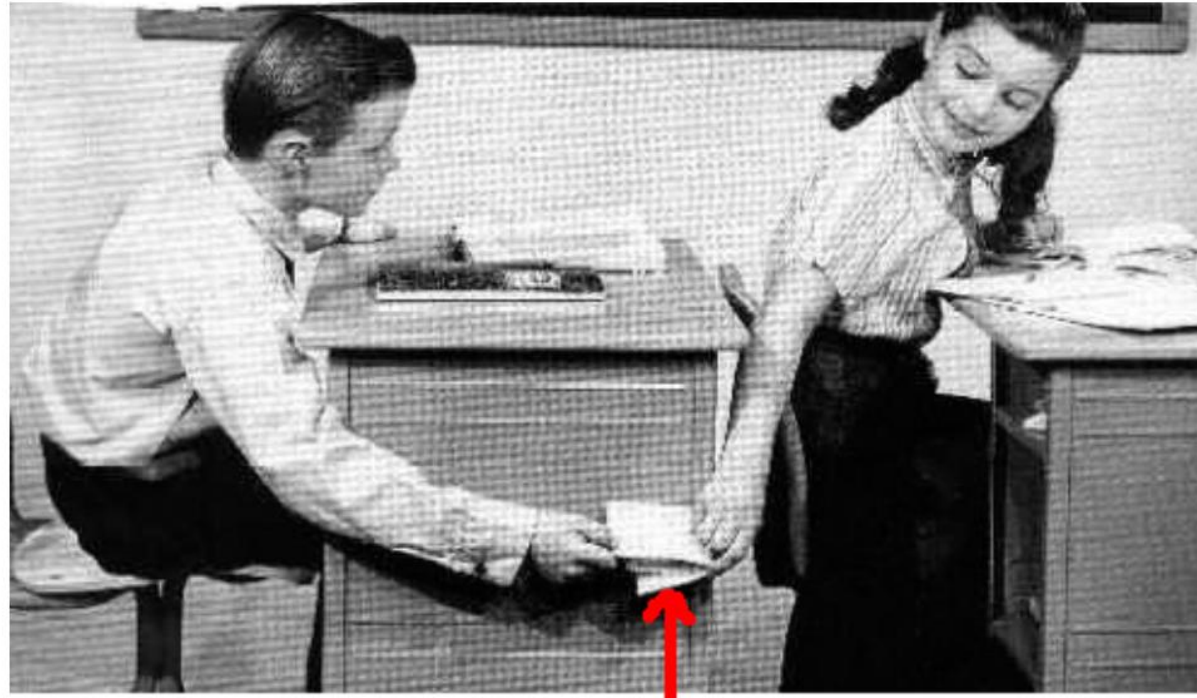
Data Stream Storage - Ejemplo

Supongamos que tiene un sitio web y cada vez que alguien carga una página, envía una notificación de evento de página vista a los consumidores. Los consumidores pueden hacer cualquiera de las siguientes cosas:

- Almacenar el mensaje en HDFS para su análisis futuro
- Contar las vistas de página y actualizar un panel
- Activar una alerta si falla una vista de página
- Enviar una notificación por correo electrónico a otro usuario

Data Stream Storage – Posible solución

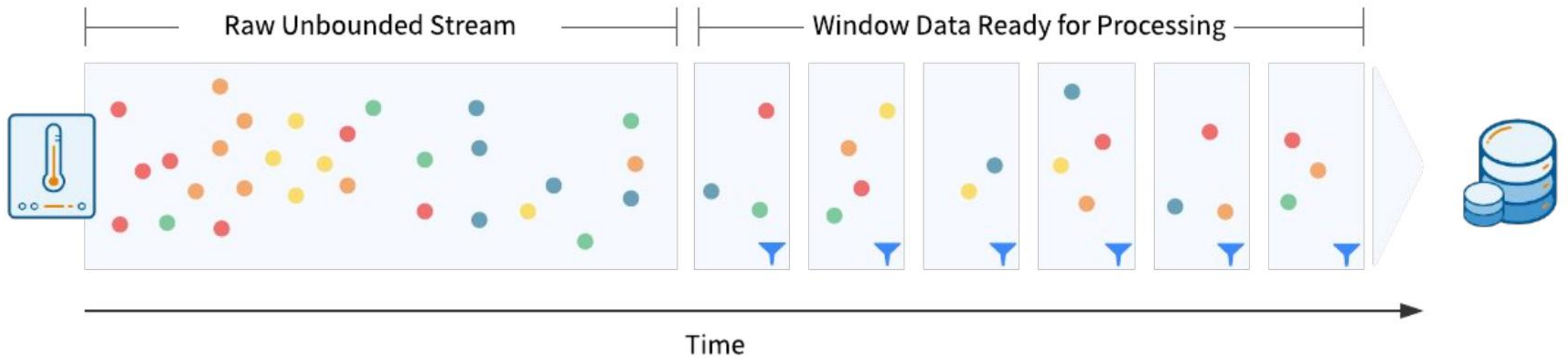
Sistemas de mensajería:



Message

www.defit.org

Data Stream Storage



Data Stream Storage – Sistemas de mensajería

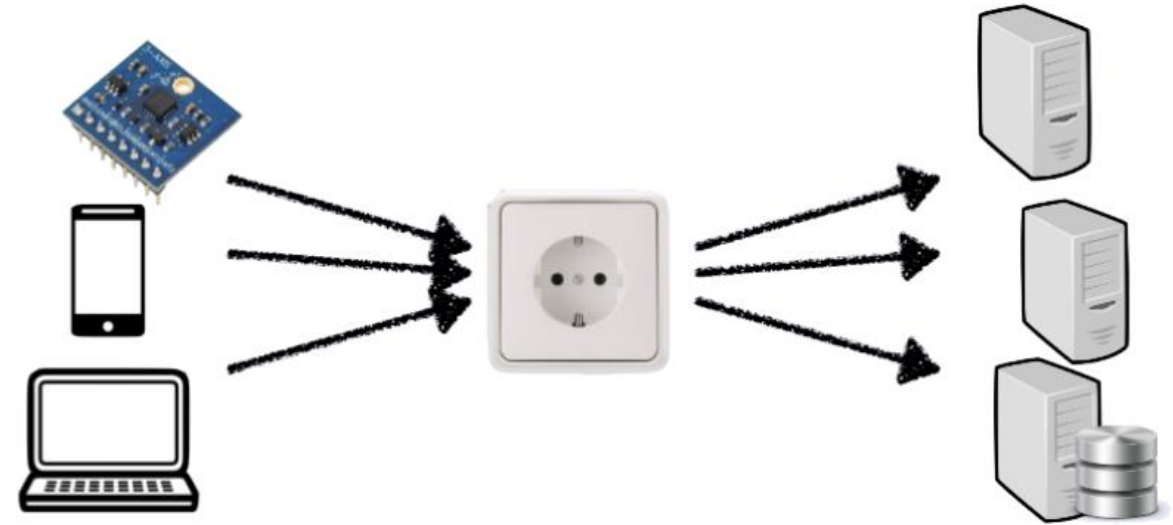
El sistema de mensajería es un enfoque para notificar a los consumidores sobre nuevos eventos.

Sistemas de mensajería

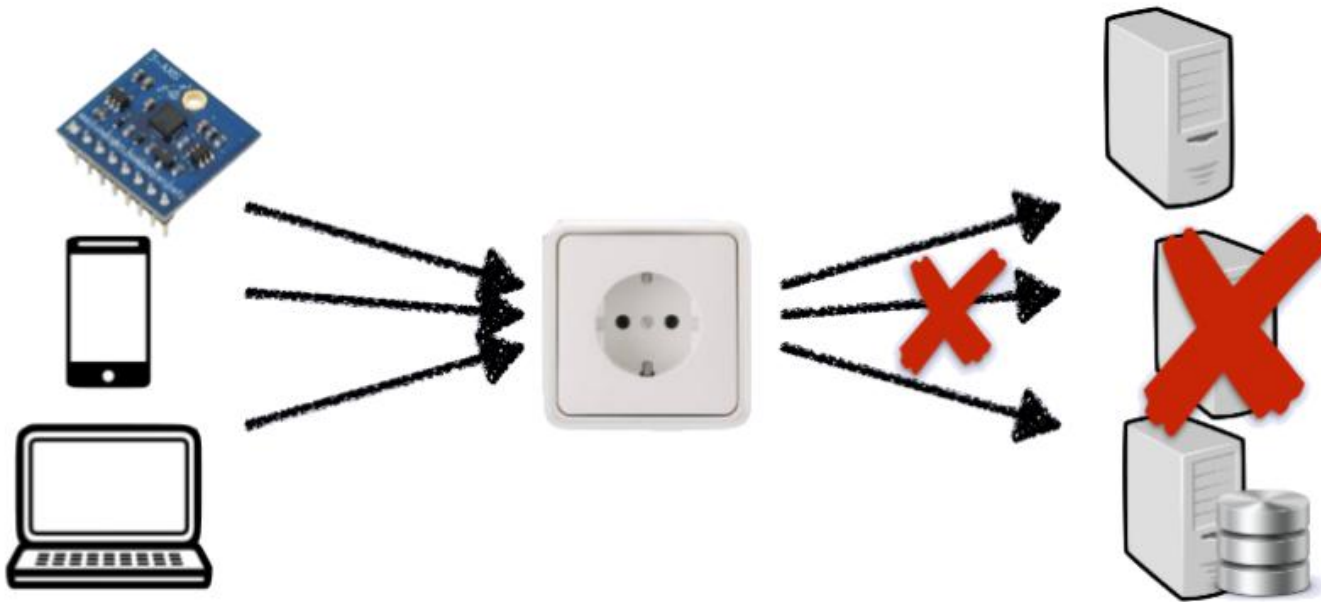
- Mensajería directa
- Agentes de mensajes (brokers)

Data Stream Storage – Mensajería directa

- Necesario en aplicaciones críticas de latencia (por ejemplo, cirugía remota).
- Un productor envía un mensaje que contiene el evento, que se envía a los consumidores.
- Tanto los consumidores como los productores tienen que estar en línea al mismo tiempo.



Data Stream Storage – Mensajería directa



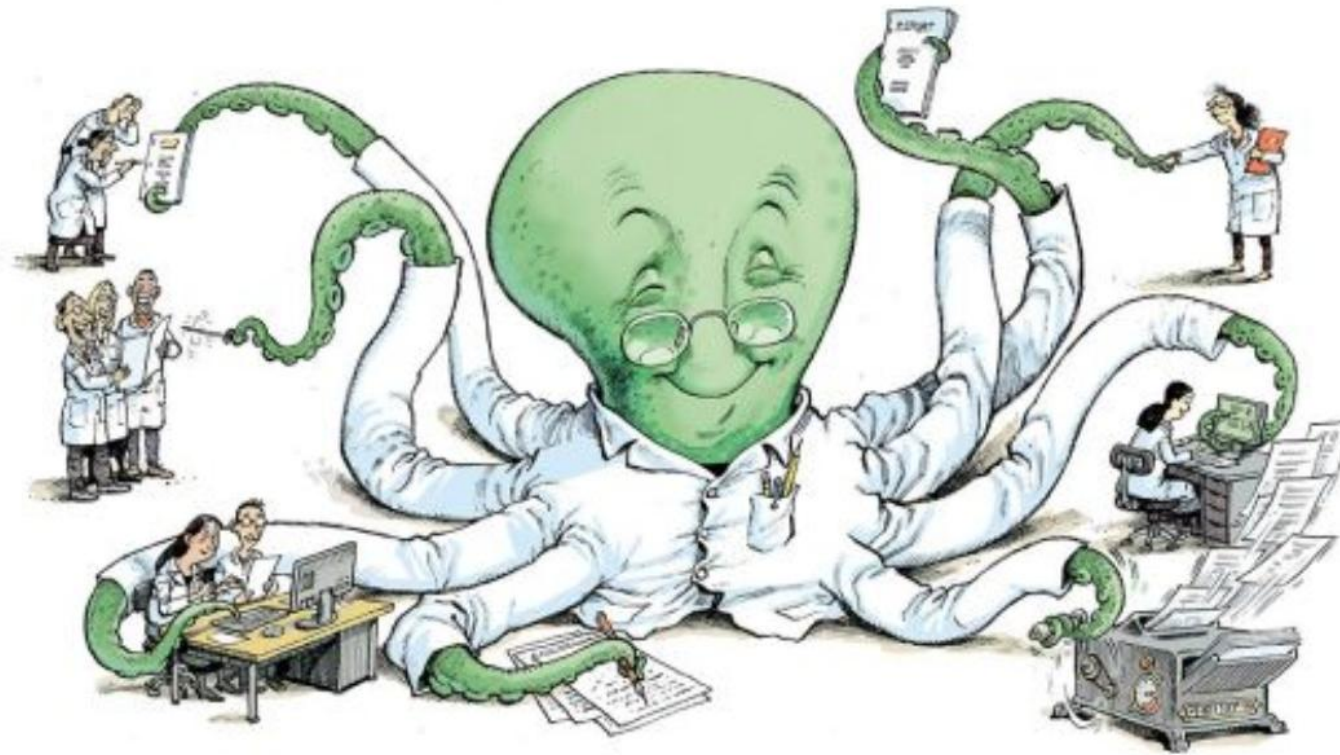
¿Qué sucede si un consumidor se bloquea o se desconecta temporalmente? (no duradero)

¿Qué sucede si los productores envían mensajes más rápido de lo que los consumidores pueden procesar?

- Eliminación de mensajes
- Saturación del sistema

Necesitamos agentes de mensajes (brokers) que puedan registrar eventos para procesarlos más adelante

Data Stream Storage – Agentes de mensajes (brokers)



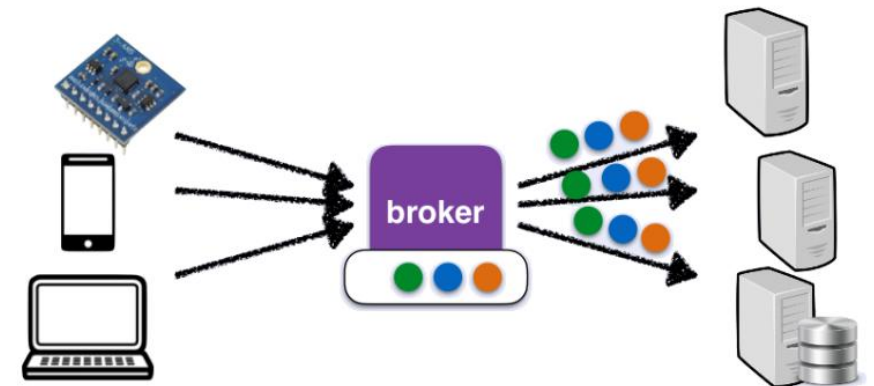
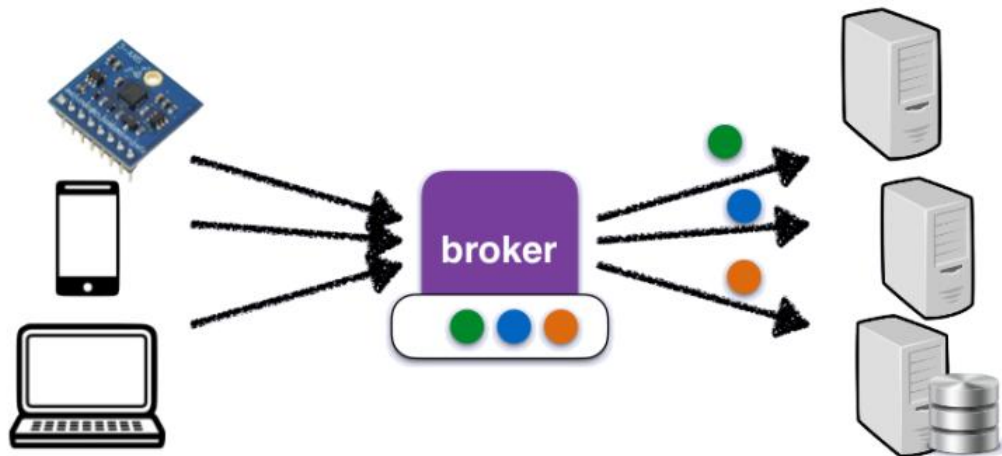
[<https://bluesyemre.com/2018/10/16/thousands-of-scientists-publish-a-paper-every-five-days>]

Agentes de mensajes (brokers)

Cuando varios consumidores leen mensajes en el mismo tema.

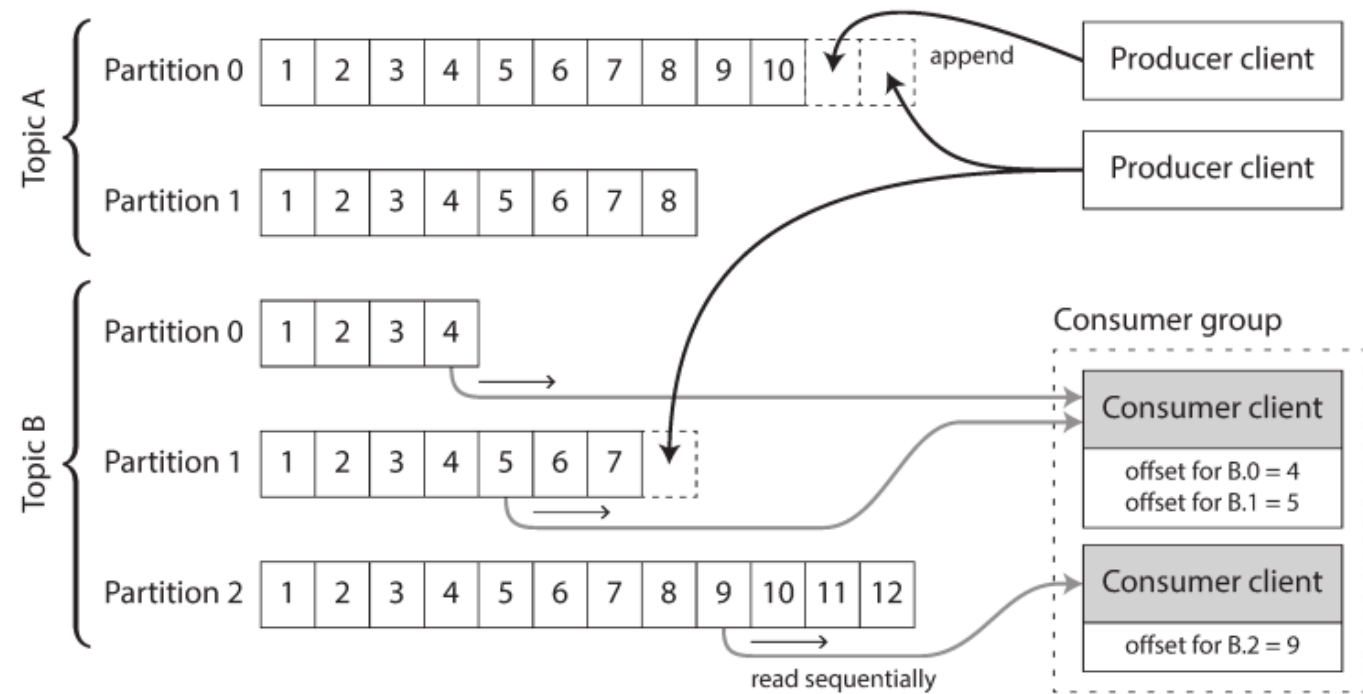
Equilibrio de carga: cada mensaje se entrega a uno de los consumidores.

Fan-out: cada mensaje se entrega a todos los consumidores.



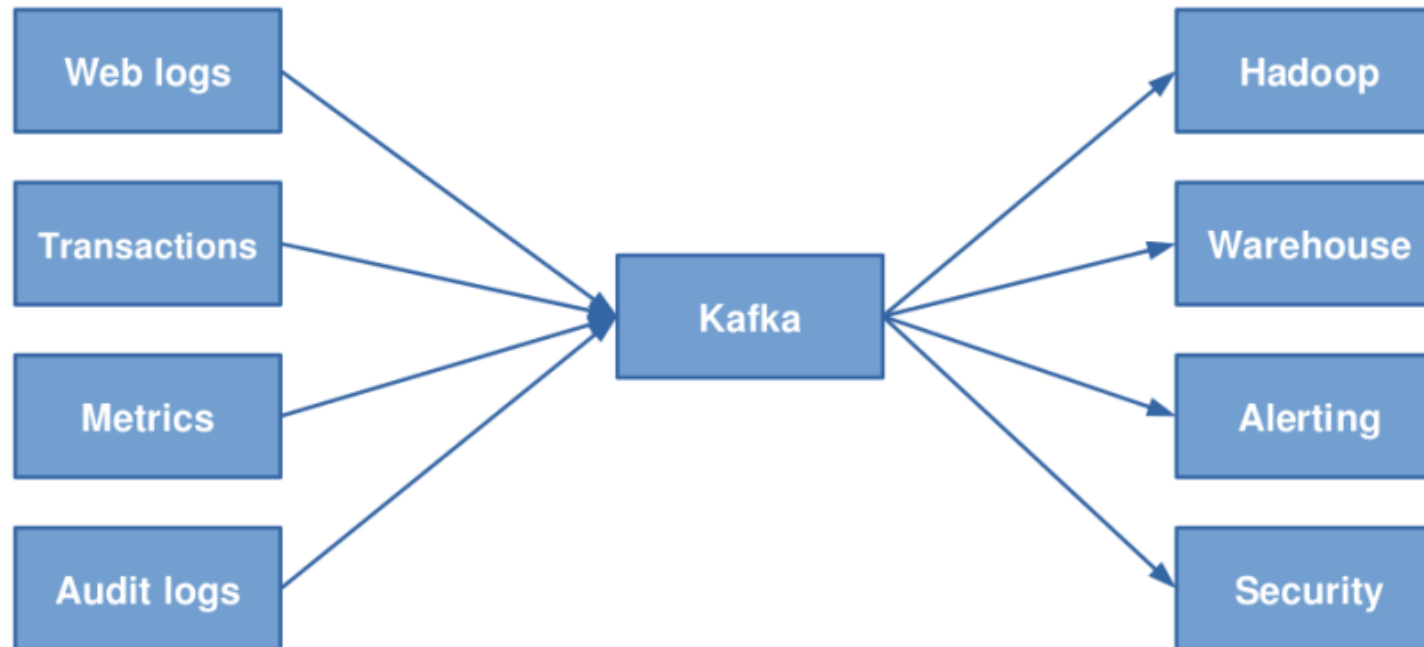
Logs particionados

- Para escalar verticalmente el sistema, los registros se pueden particionar y guardar en diferentes máquinas.
- Cada partición se puede leer y escribir independientemente de las demás.
- Un topic es un grupo de particiones que transportan mensajes del mismo tipo.
- Dentro de cada partición, el intermediario asigna un número de secuencia monótonamente creciente (desplazamiento) a cada mensaje
- Sin garantía de ordenación entre particiones



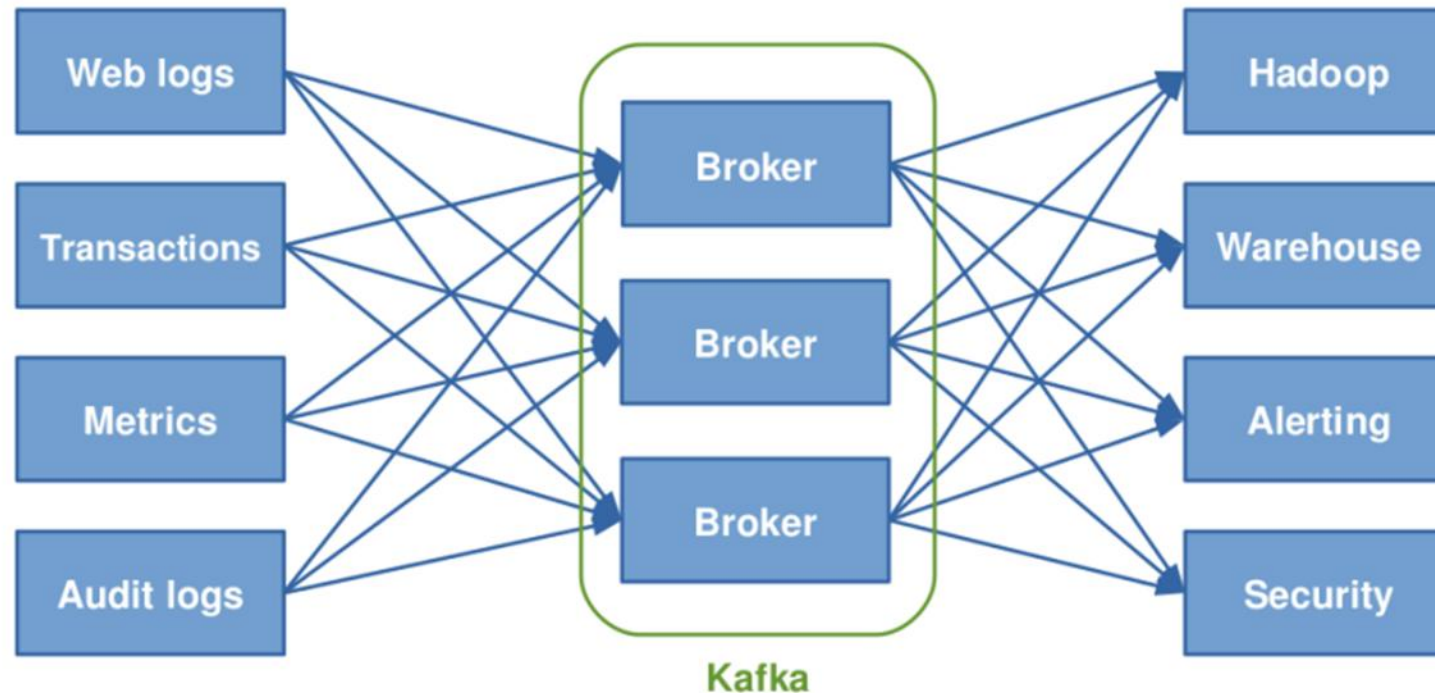
Kafka - A Log-Based Message Broker

Kafka es un servicio de registros distribuido, orientado a temas (tópicos), particionado y replicado.



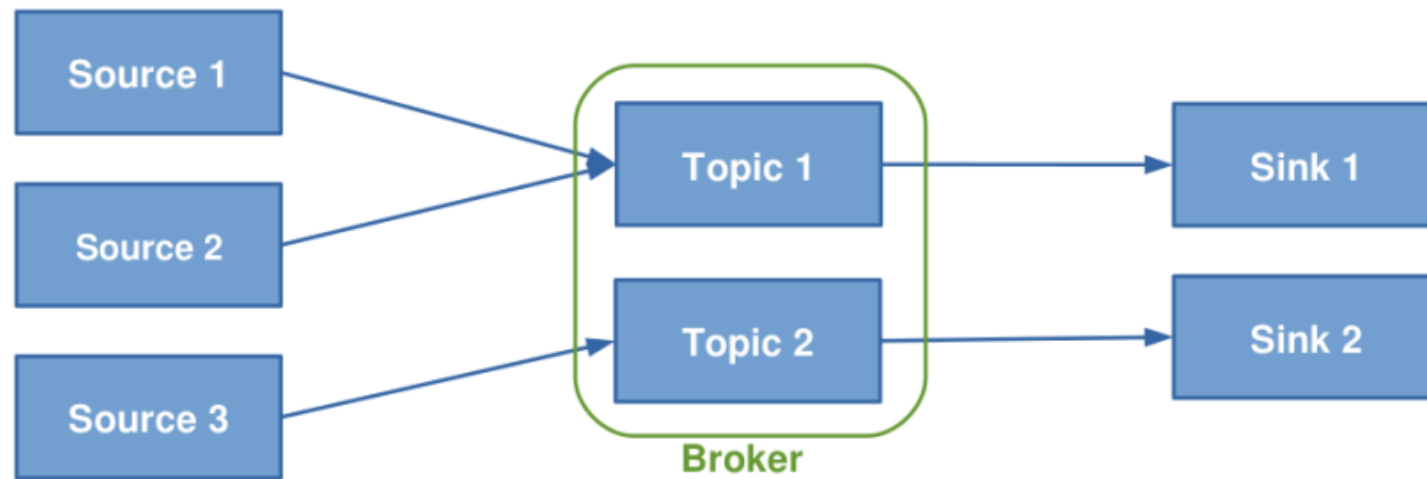
Kafka - A Log-Based Message Broker

Kafka es un servicio de registros **distribuido**, orientado a temas (tópicos), particionado y replicado.



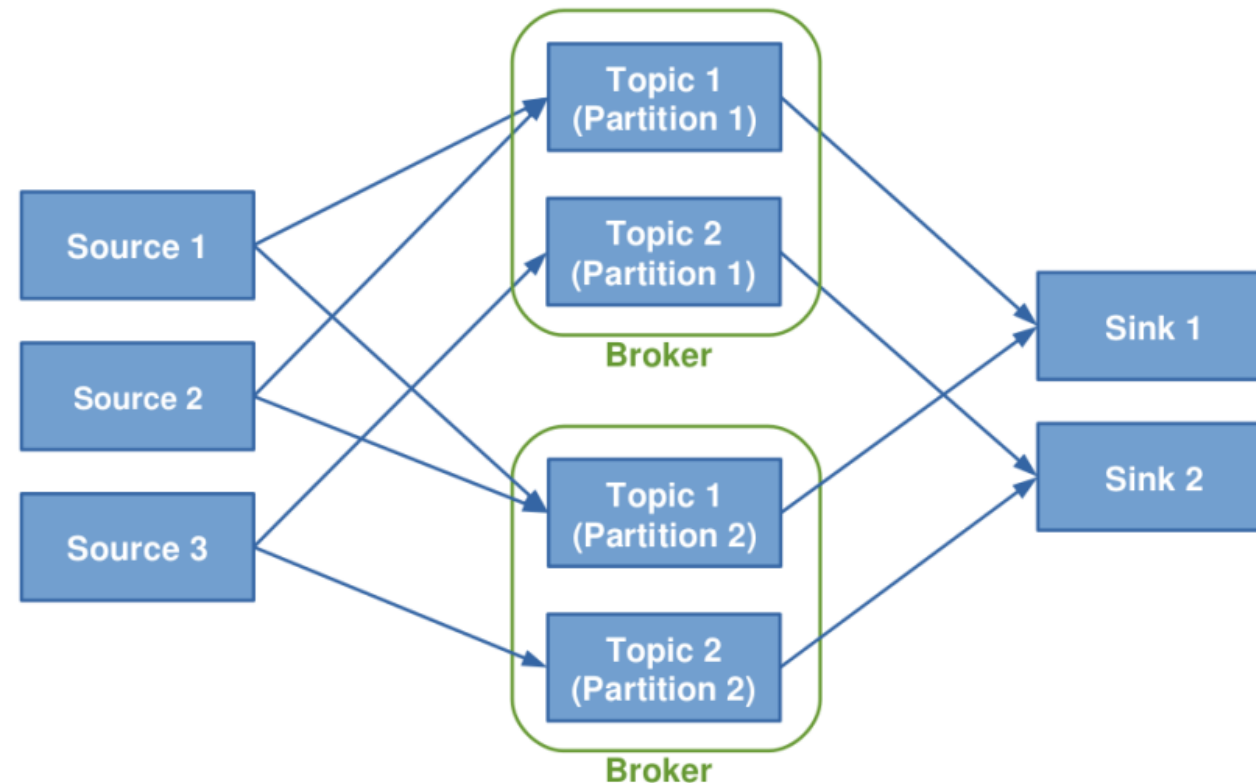
Kafka - A Log-Based Message Broker

Kafka es un servicio de registros **distribuido**, **orientado a temas (tópicos)**, particionado y replicado.



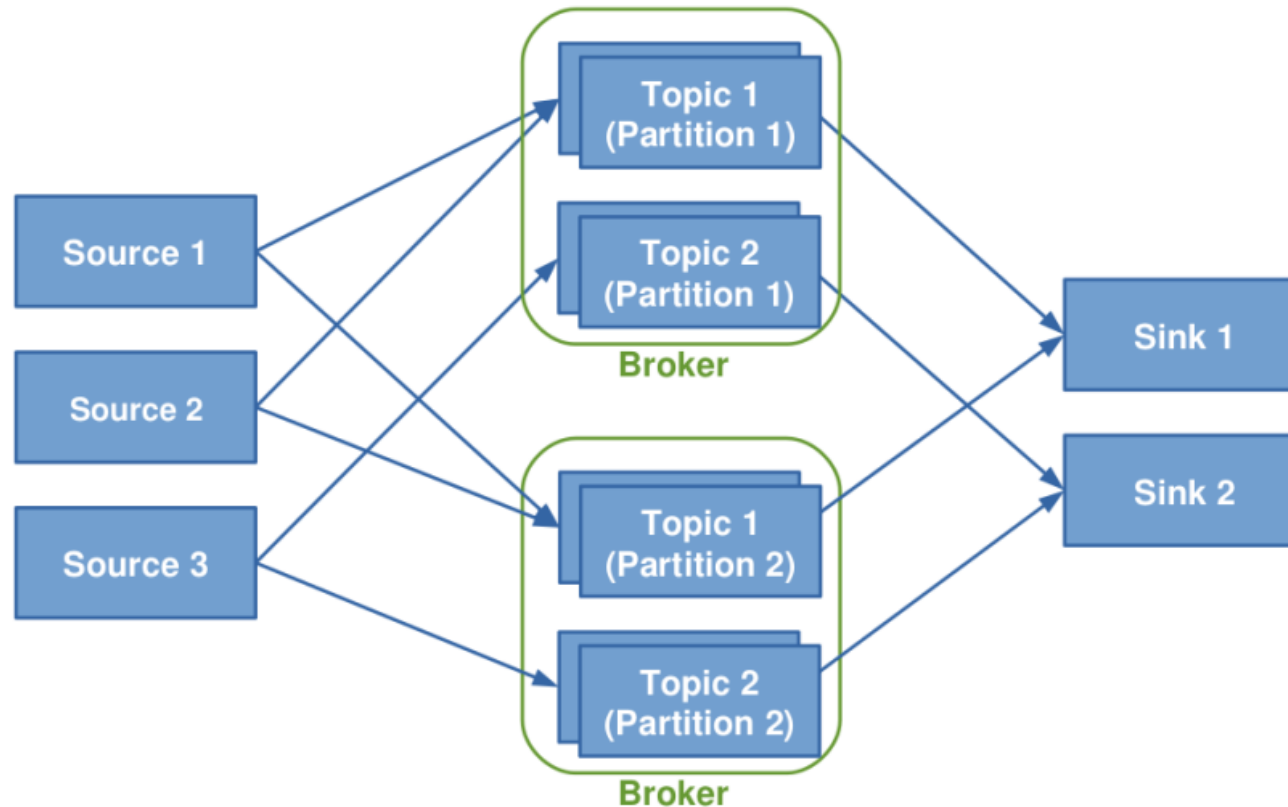
Kafka - A Log-Based Message Broker

Kafka es un servicio de registros **distribuido**, **orientado a temas (tópicos)**, **particionado** y replicado.



Kafka - A Log-Based Message Broker

Kafka es un servicio de registros **distribuido, orientado a temas (tópicos), particionado y replicado.**



Logs, Topics, Partitions

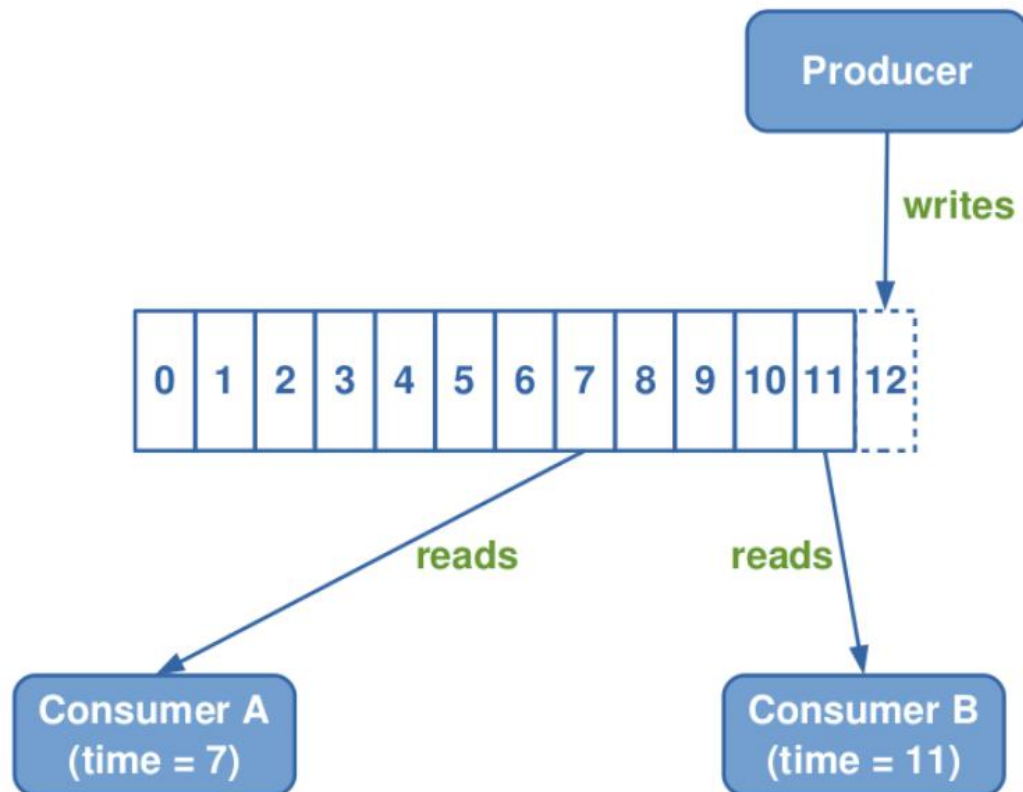
- Kafka trabaja con Logs.
- Topics son queues: un flujo de mensajes de un tipo particular

```
jkreps-mn:~ jkreps$ tail -f -n 20 /var/log/apache2/access_log
::1 - - [23/Mar/2014:15:07:00 -0700] "GET /images/apache_feather.gif HTTP/1.1" 200 4128
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/producer_consumer.png HTTP/1.1" 200 8f
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/log_anatomy.png HTTP/1.1" 200 19579
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/consumer-groups.png HTTP/1.1" 200 2682
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/log_compaction.png HTTP/1.1" 200 4141
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /documentation.html HTTP/1.1" 200 189893
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/log_cleaner_anatomy.png HTTP/1.1" 200
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/kafka_log.png HTTP/1.1" 200 134321
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/mirror-maker.png HTTP/1.1" 200 17054
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /documentation.html HTTP/1.1" 200 189937
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /styles.css HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/kafka_logo.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/producer_consumer.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/log_anatomy.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/consumer-groups.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/log_cleaner_anatomy.png HTTP/1.1" 304
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/log_compaction.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/kafka_log.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/mirror-maker.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:09:55 -0700] "GET /documentation.html HTTP/1.1" 200 195264
```



Logs, Topics, Partitions

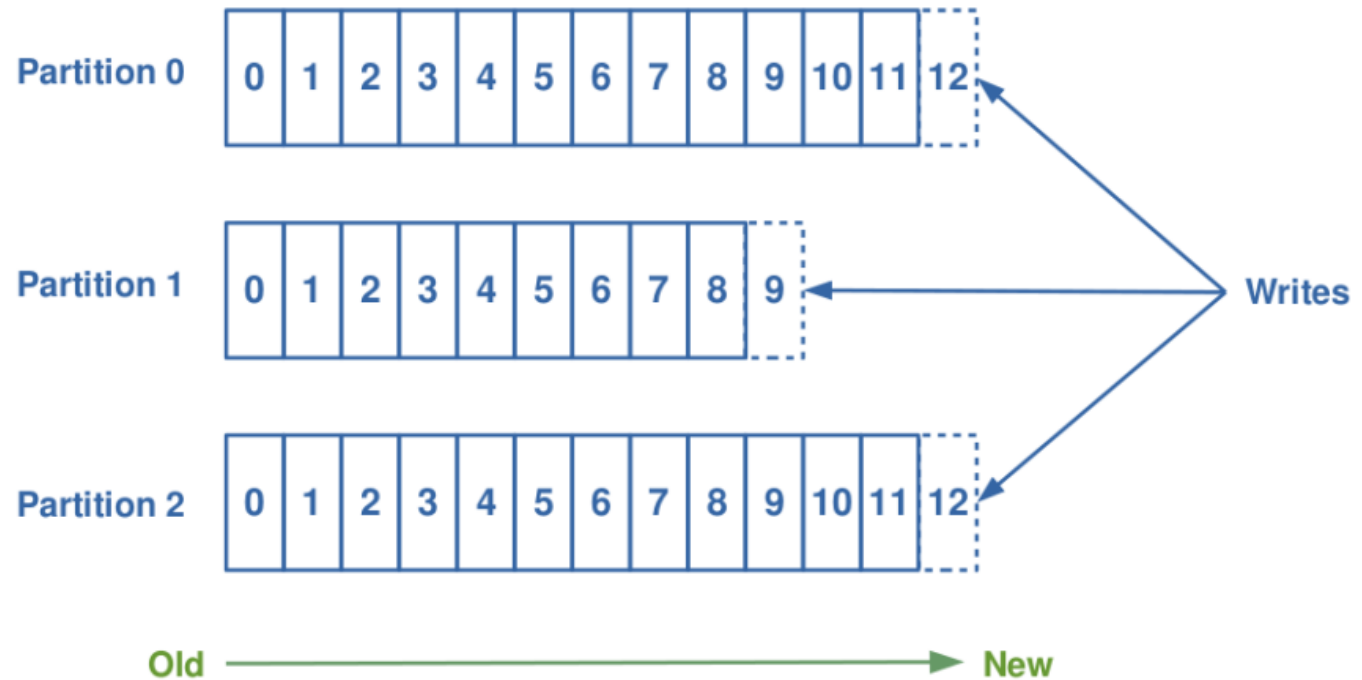
- A cada mensaje se le asigna un identificador secuencial llamado offset.



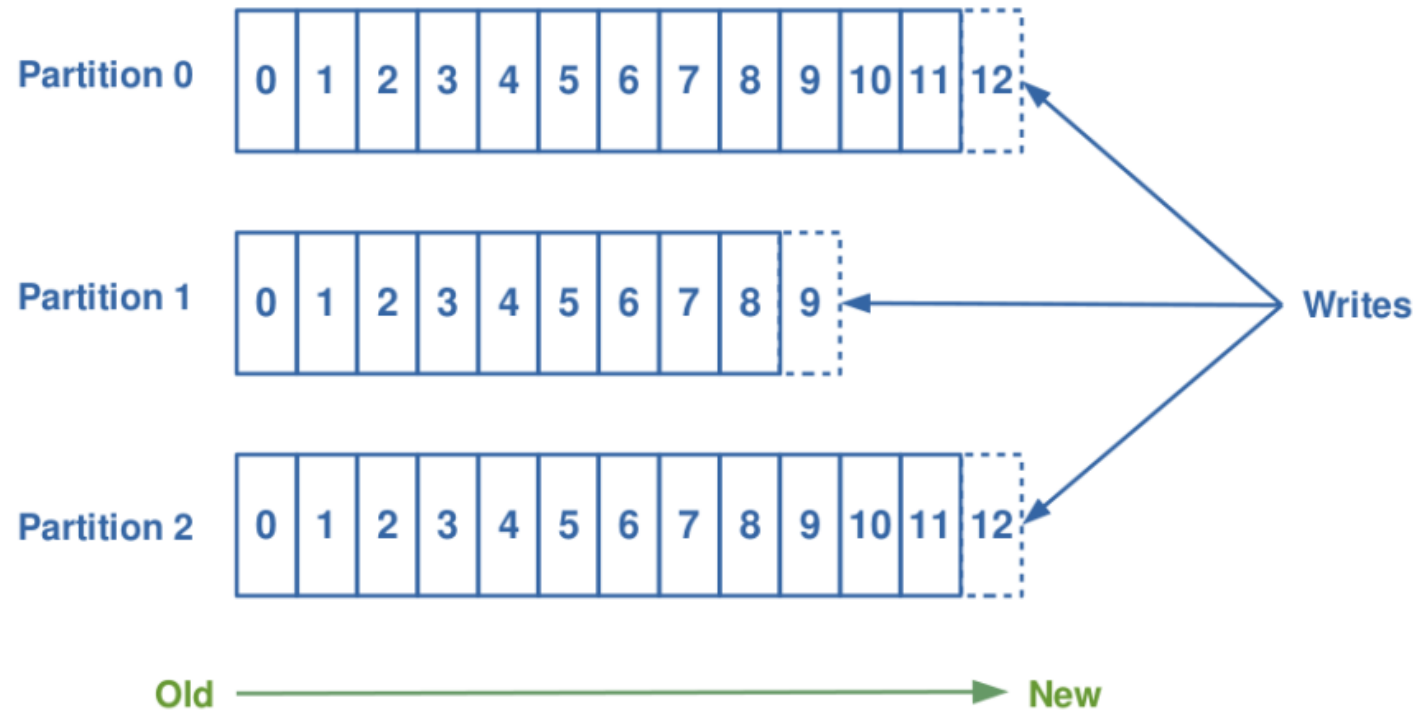
Logs, Topics, Partitions

Los Topics son colecciones lógicas de particiones (los archivos físicos).

- Ordenado
- Solo agregación (append-only)
- Inmutable



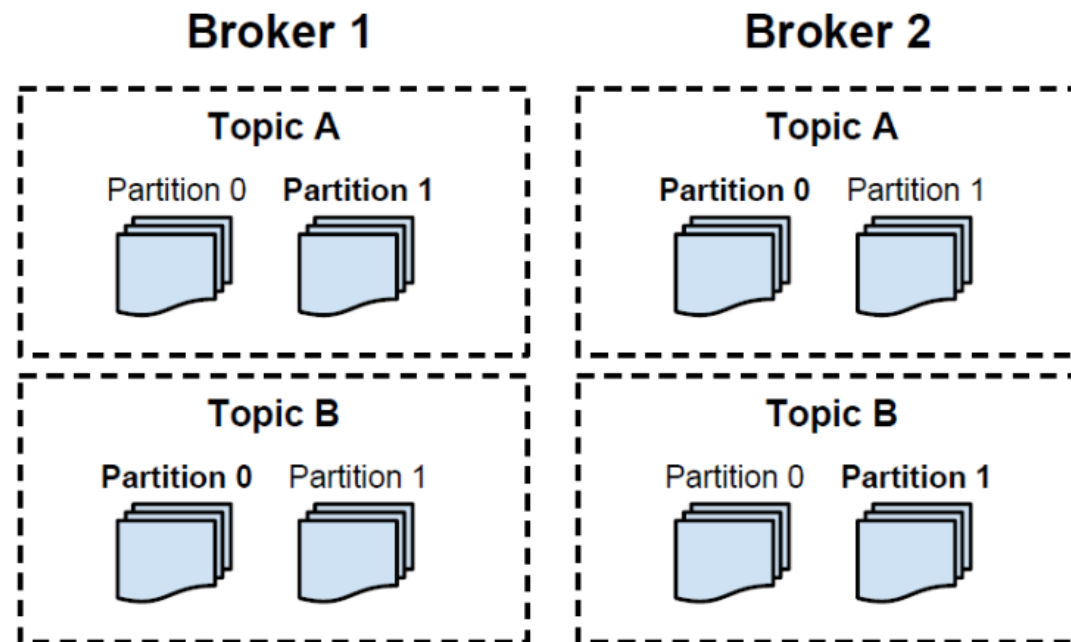
Logs, Topics, Partitions



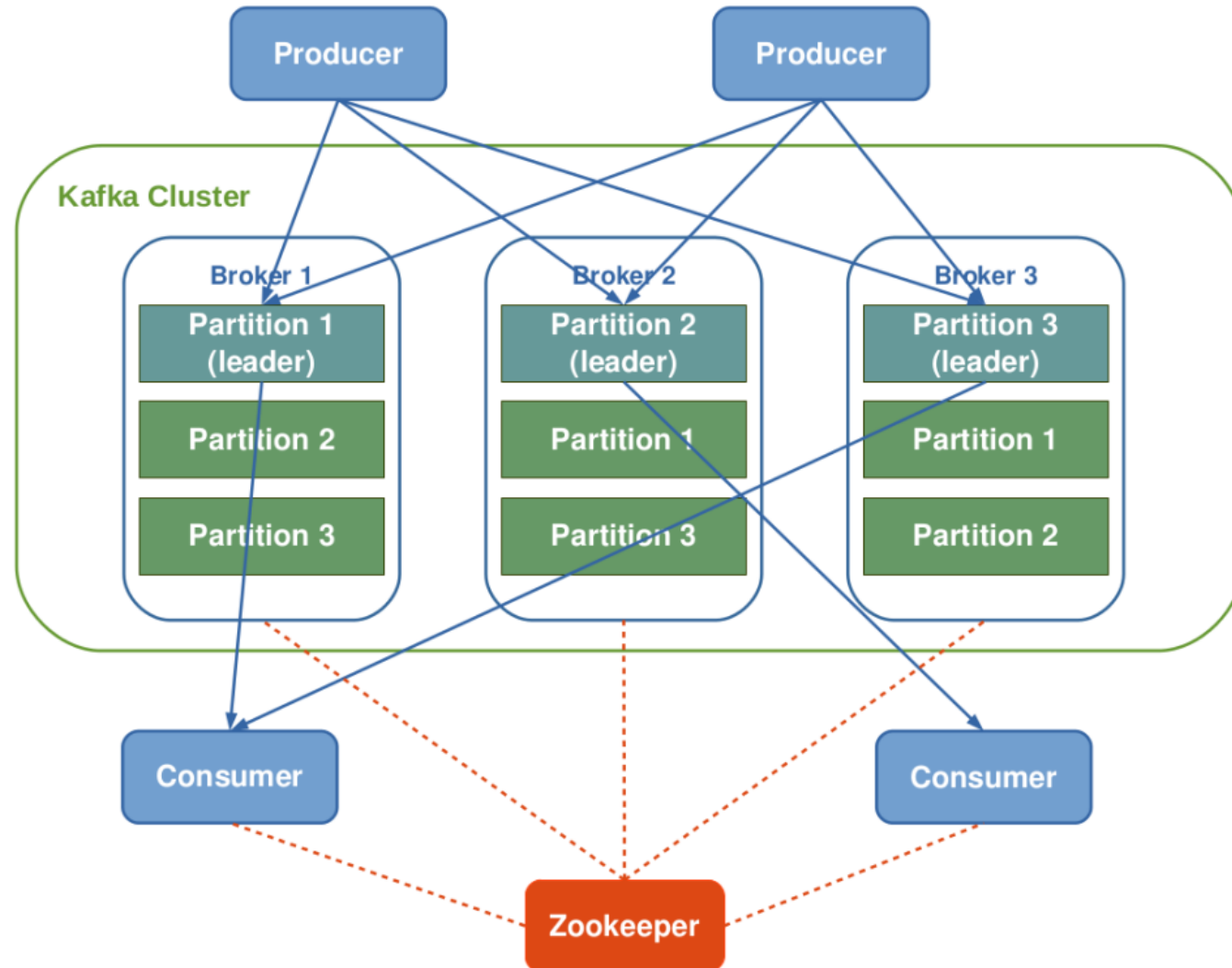
- El orden solo se garantiza dentro de una partición para un topico.
- Los mensajes enviados por un productor a una partición del topico determinada se agregarán en el orden en que se envían.
- Una instancia de consumidor ve los mensajes en el orden en que se almacenan en el registro.

Logs, Topics, Partitions

- Las particiones de un tema se replican: tolerancia a errores
- Un broker contiene algunas de las particiones de un tema.
- Un broker es el líder de una partición: todas las escrituras y lecturas deben ir al líder.



Arquitectura Kafka



Arquitectura Kafka

- Kafka usa "Zookeeper" para las siguientes tareas:
- Detectar la adición y la eliminación de brokers y consumidores.
- Realizar un seguimiento del offset consumido de cada partición.

Estados Kafka

- Los brokers no tienen estado: no hay metadatos para los consumidores-productores en los brokers.
- Los consumidores son responsables de realizar un seguimiento de los offsets.
- Los mensajes en las colas caducan en función de períodos de tiempo preconfigurados (por ejemplo, una vez al día).

Garantías del envío

- Kafka garantiza que los mensajes de una sola partición se entreguen a un consumidor en orden.
- No hay ninguna garantía sobre el orden de los mensajes procedentes de diferentes particiones.
- Kafka solo garantiza la entrega al menos una vez

Corriendo Kafka

```
# Start the ZooKeeper
```

```
zookeeper-server-start.sh config/zookeeper.properties
```

```
# Start the Kafka server
```

```
kafka-server-start.sh config/server.properties
```

```
# Create a topic, called "avg"
```

```
kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1  
--topic avg
```

```
# Produce messages and send them to the topic "avg"
```

```
kafka-console-producer.sh --broker-list localhost:9092 --topic avg
```

```
# Consume the messages sent to the topic "avg"
```

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic avg --from-beginning
```

Data Stream Processing

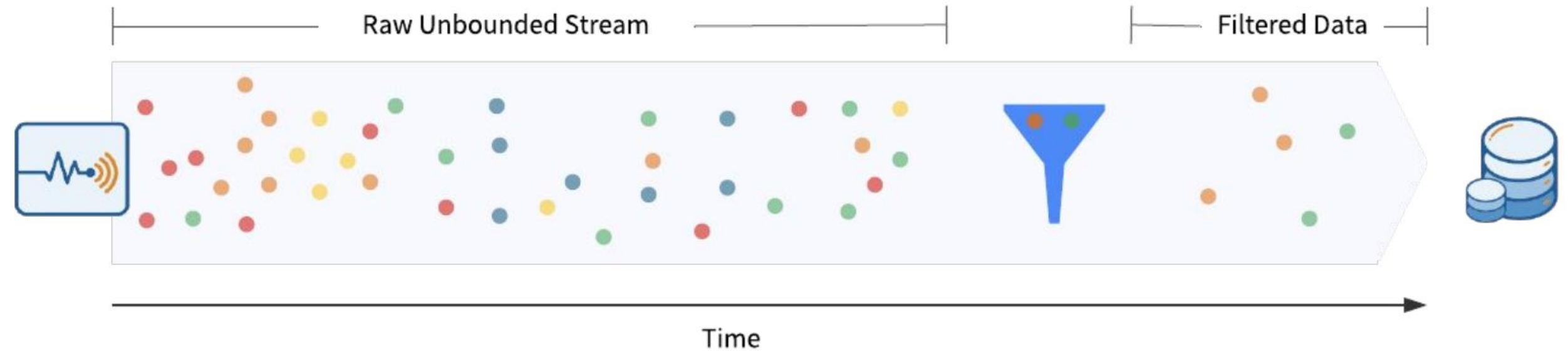
- El flujo de datos son datos no enlazados, que se dividen en una secuencia de tuplas individuales.
- Una tupla de datos es el elemento de datos atómicos de un flujo de datos.
- Puede ser estructurado, semiestructurado y no estructurado

Operaciones

- Filtrado
- Enriquecimiento
- Agregación
- Detección de eventos

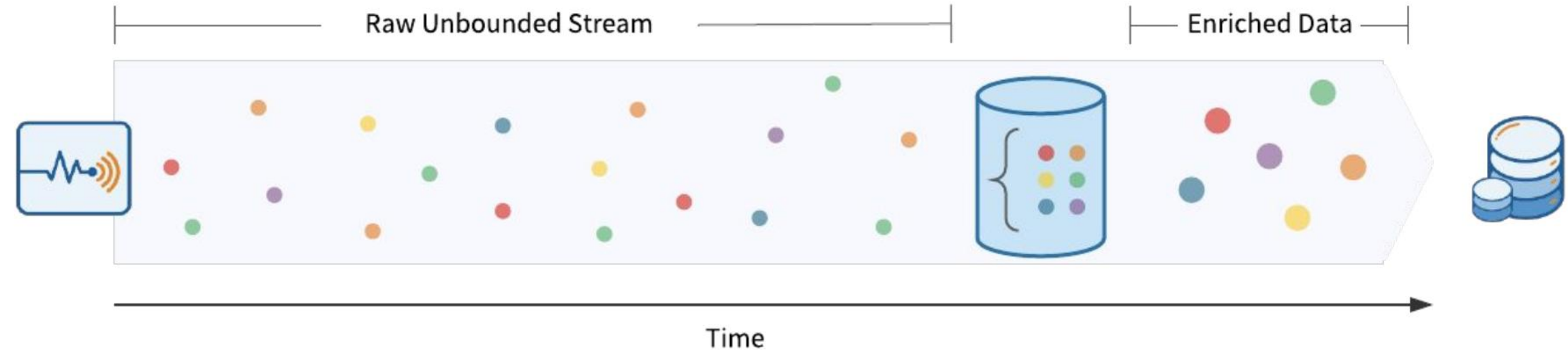
Data Stream Processing - Filtrado

- Reduce los volúmenes de datos en la memoria antes de confirmar los datos en el disco.



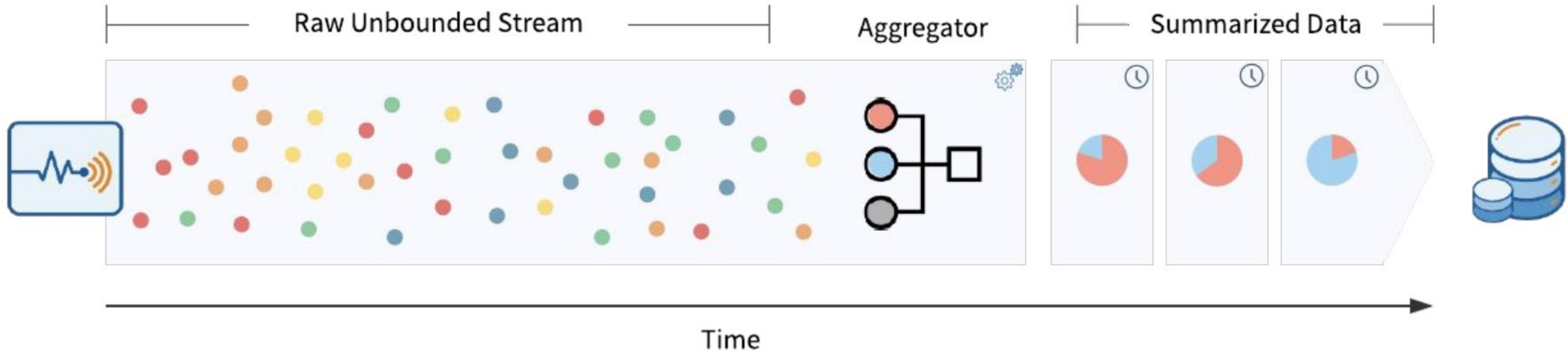
Data Stream Processing - Enriquecimiento

Une los datos no enlazados a otros conjuntos de datos (bases de datos, API) antes de confirmar los datos en el disco.



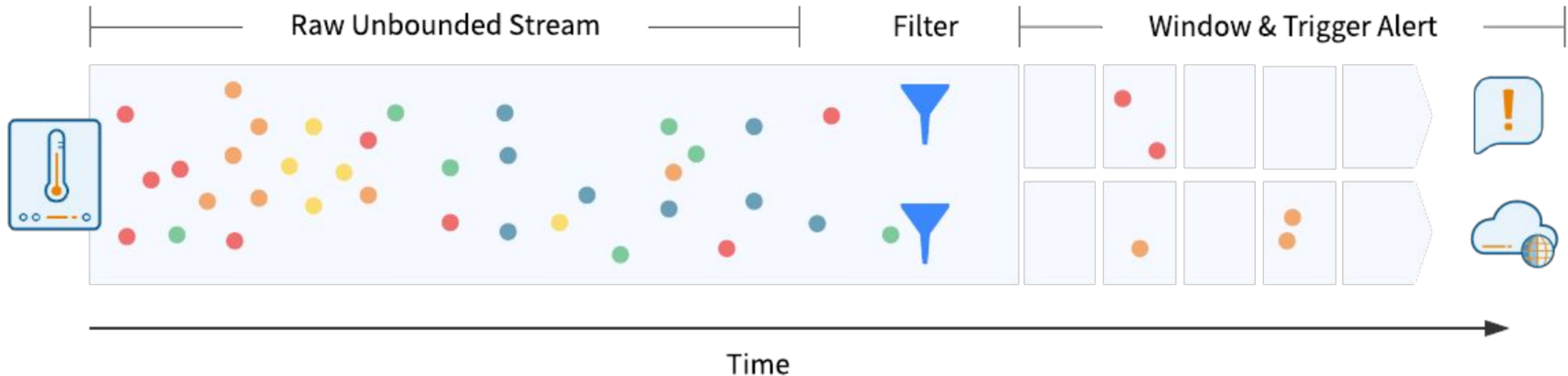
Data Stream Processing - Agregación

Resume los datos no enlazados calculando agregaciones de ventana de tiempo antes de confirmar los datos en el disco.



Data Stream Processing – Detección de eventos

Detecte patrones en la memoria y luego active un evento cuando se cumplan ciertos criterios.



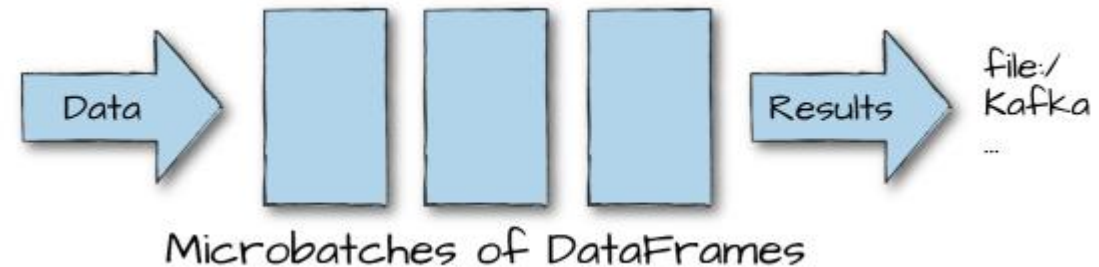
Data Stream Processing – Puntos claves

- Procesamiento continuo frente a microlotes (micro-batch)
- APIs declarativas vs Record-at-a-Time
- Tiempo de evento vs. tiempo de procesamiento
- Windowing

Data Stream Processing – Puntos claves

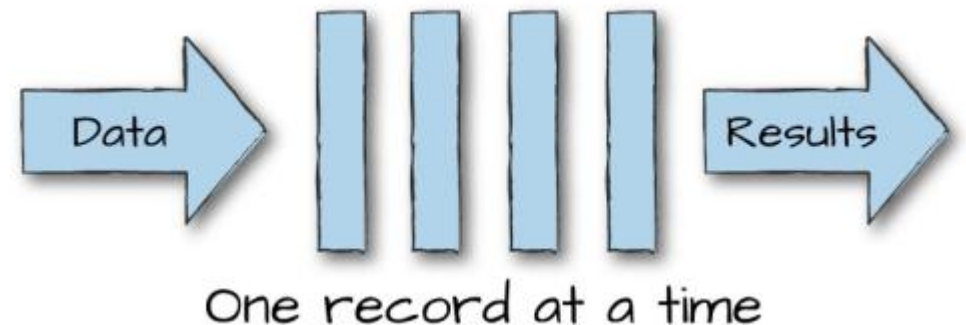
Sistemas de microlotes

- Motores por lotes
- Dividir los datos no enlazados en un conjunto de datos delimitados y, a continuación, procesar cada lote.



Sistemas basados en el procesamiento continuo

- Cada nodo del sistema escucha continuamente los mensajes de otros nodos y genera nuevas actualizaciones en sus nodos secundarios.



Data Stream Processing – Puntos claves

API de grabación a la vez (por ejemplo, Storm)

- API de bajo nivel
- Pasa cada evento a la aplicación y deja que reaccione.
- Útil cuando las aplicaciones necesitan un control total sobre el procesamiento de datos.
- Los factores complicados, como el mantenimiento del estado, se rigen por la aplicación.

API declarativa (por ejemplo, Spark streaming, Flink, Google Dataflow)

- Las aplicaciones especifican qué calcular, no cómo calcularlo en respuesta a cada nuevo evento.

Data Stream Processing – Puntos claves

Tiempo del evento: la hora en la que realmente ocurrieron los eventos. Marcas de tiempo insertadas en cada registro en el origen.

Tiempo de procesamiento: el momento en que se recibe el registro en la aplicación de transmisión.

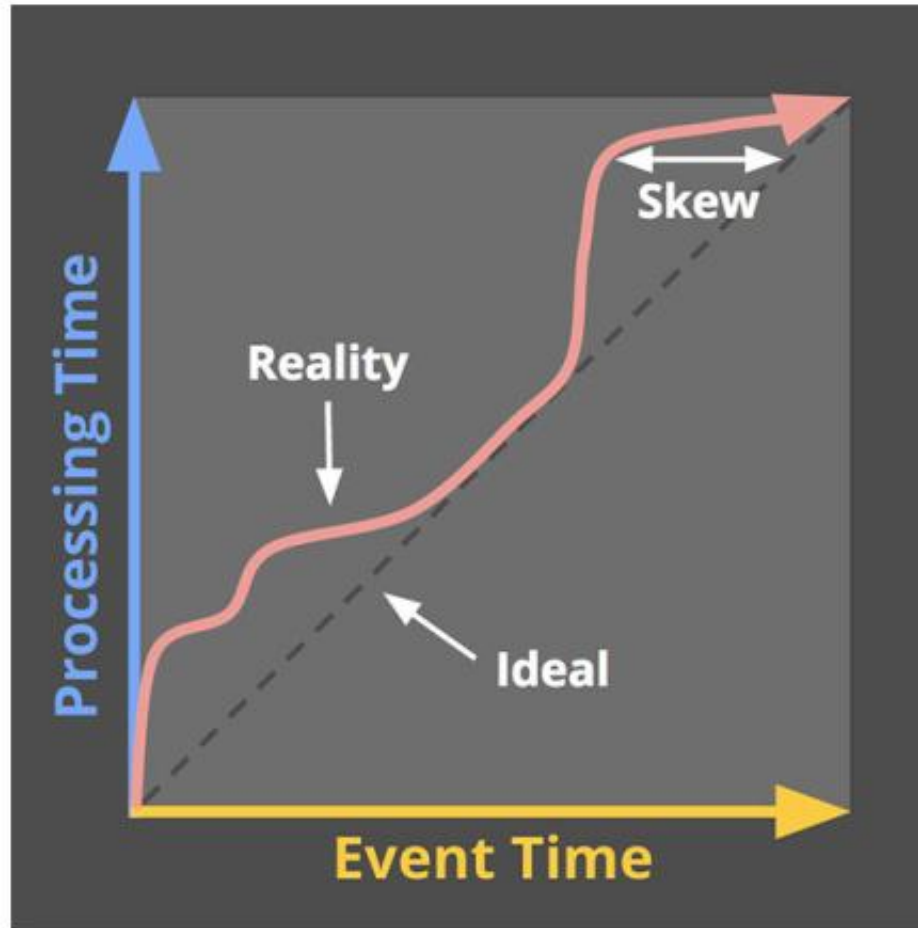
- Idealmente, el tiempo del evento y el tiempo de procesamiento deben ser iguales.
- Sesgo entre el tiempo del evento y el tiempo de procesamiento.

Data Stream Processing – Puntos claves

Tiempo
de tiem

Tiempo
aplicaci

- Idealr
iguale
- Sesgo



✓ Marcas

3

1 ser

[<https://www.oreilly.com/ideas/the-world-beyond-batch-streaming-101>]

Data Stream Processing – Puntos claves

Windowing: un búfer asociado con un puerto de entrada para retener las tuplas recibidas anteriormente. Cuatro directivas de administración de ventanas diferentes.

- Directiva basada en recuento: el número máximo de tuplas que puede contener un búfer de ventana
- Directiva basada en delta: un umbral delta en un atributo de tupla
- Política basada en la puntuación: se recibe una puntuación
- Directiva basada en el tiempo: basada en el período de tiempo de procesamiento o evento

Data Stream Processing – Puntos claves

Dos tipos de ventanas: tumbling y sliding

- Ventana de saltos de tamaño constante (tumbling): admite operaciones por lotes. Cuando el búfer se llena, se expulsan todas las tuplas.



- Ventana deslizante (sliding): admite operaciones incrementales. Cuando el búfer se llena, se expulsan las tuplas más antiguas.

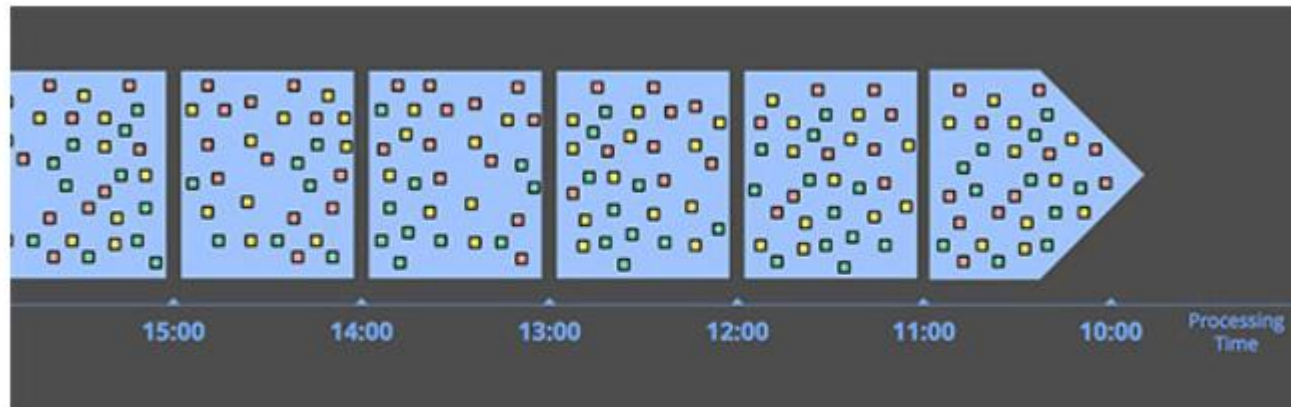


Data Stream Processing – Puntos claves

Ventanas por tiempo de procesamiento:

El sistema almacena en búfer los datos entrantes en las ventanas hasta que el tiempo de procesamiento ha pasado.

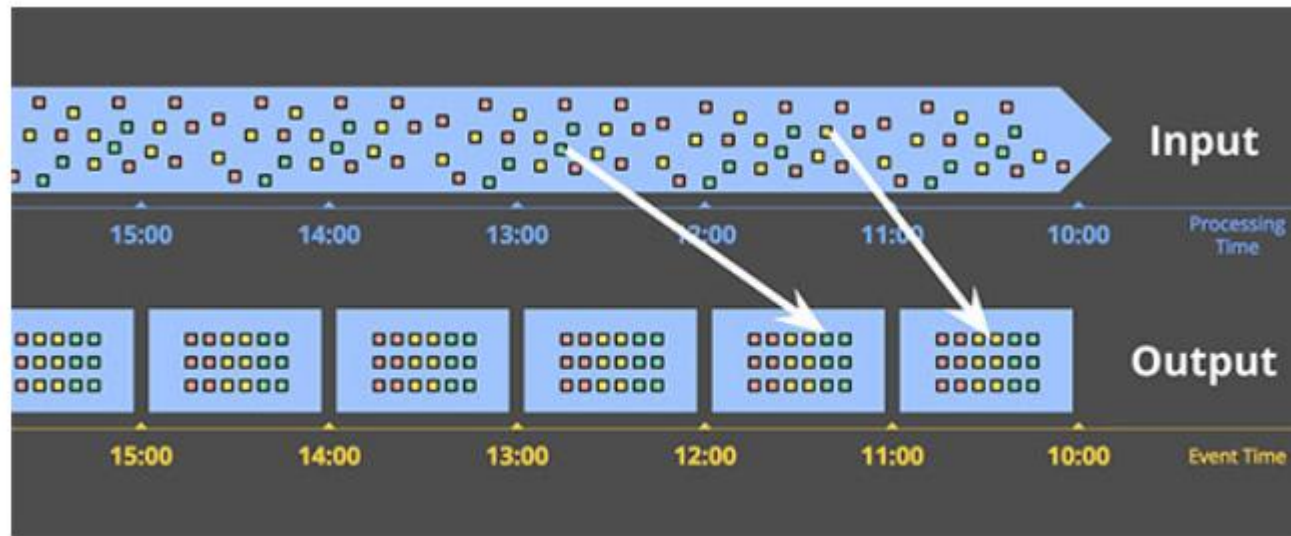
Por ejemplo, ventanas fijas de cinco minutos



[<https://www.oreilly.com/ideas/the-world-beyond-batch-streaming-101>]

Data Stream Processing – Puntos claves

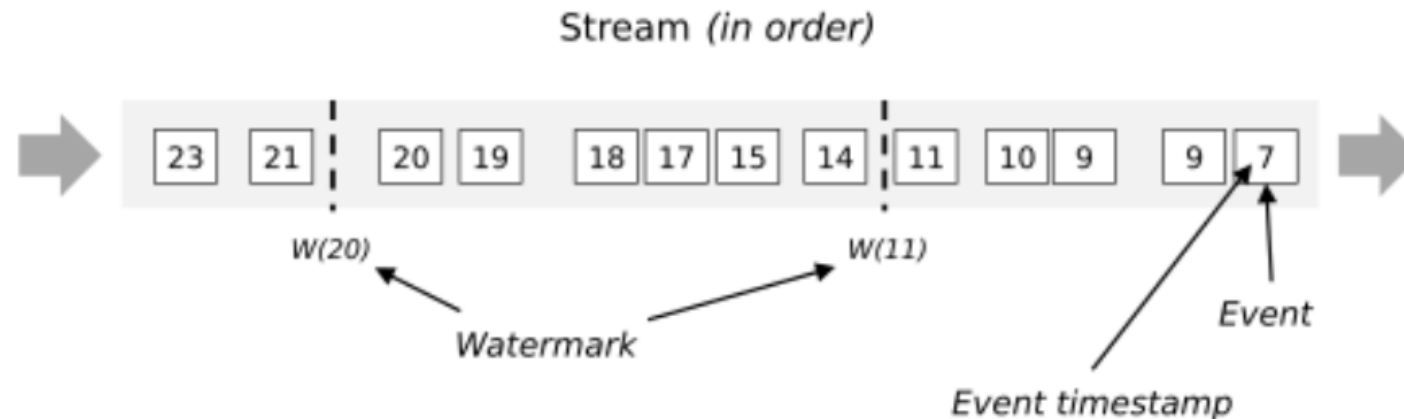
Ventanas por tiempo del evento: Trabaja sobre los momentos en que realmente ocurrieron los eventos. Manejo de eventos fuera de orden.



[<https://www.oreilly.com/ideas/the-world-beyond-batch-streaming-101>]

Data Stream Processing – Puntos claves

- La marca de agua (watermarking) ayuda a un sistema de procesamiento de streaming a lidiar con la latencia.
- Las marcas de agua hacen parte del flujo de datos y llevan una marca de tiempo t .
- Una marca de agua es un umbral para especificar cuánto tiempo espera el sistema los eventos tardíos.
- Los sistemas de streaming utilizan marcas de agua para medir el progreso en el tiempo del evento.



Data Stream Processing – Puntos claves

Un $W(t)$ declara que el tiempo del evento ha alcanzado el tiempo t en ese flujo

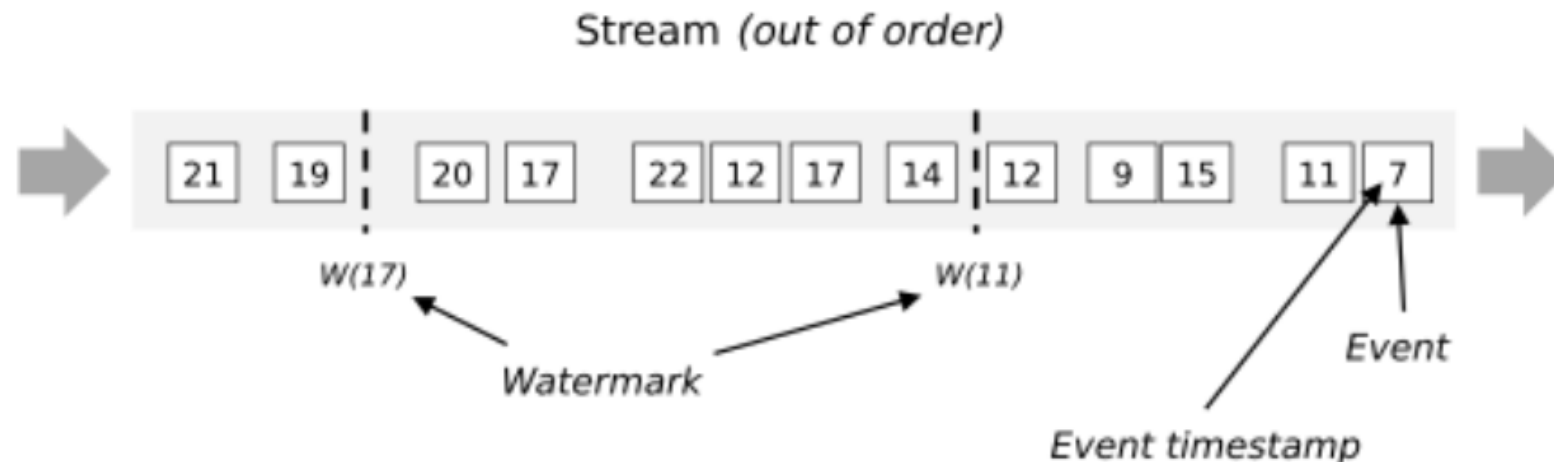
No debe haber más elementos de la secuencia con una marca de tiempo $t_0 \leq t$.

Es posible que ciertos elementos violen la condición de marca de agua.

Después de que se haya producido el $W(t)$, se producirán más elementos con marca de tiempo $t_0 \leq t$.

Si un evento entrante se encuentra dentro de la marca de agua, se usa para actualizar una consulta.

Los programas de transmisión pueden esperar explícitamente algunos elementos tardíos



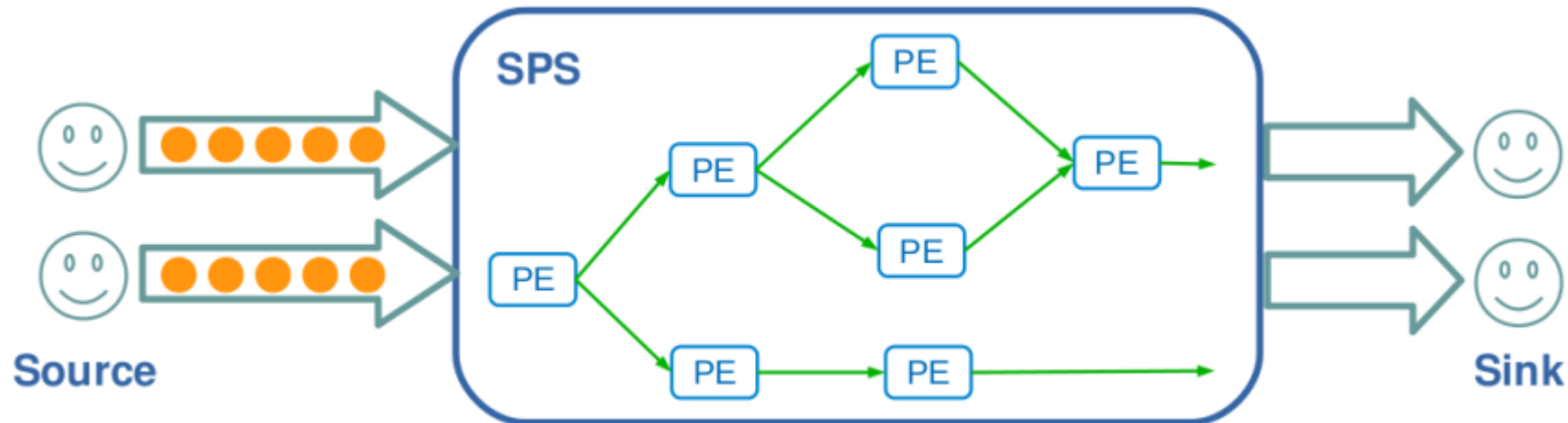
Data Stream Processing – Modelo

Las tuplas son procesadas por los operadores de la aplicación o el elemento de procesamiento (PE).

Un PE es la unidad funcional básica en una aplicación.

Un PE procesa tuplas de entrada, aplica una función y genera tuplas.

Un conjunto de PE y conexiones de flujo, organizadas en un gráfico de flujo de datos



Data Stream Processing – Modelo

Un PE puede mantener el estado interno en las tuplas mientras las procesa, o procese las tuplas independientemente unas de otras.

Tareas con estado vs tareas sin estado

Data Stream Processing – Modelo

Tareas sin estado: no mantienen el estado y procesan cada tupla independientemente del historial anterior, o incluso del orden de llegada de las tuplas.

- Fácilmente paralelizable.
- No hay sincronización.
- Reiniciar en caso de errores sin necesidad de ningún procedimiento de recuperación

Data Stream Processing – Modelo

Tareas con estado: implica mantener la información en diferentes tuplas para detectar

- Patrones complejos.
- Una EP suele ser una sinopsis de las tuplas recibidas hasta el momento.
- Subconjunto de tuplas recientes guardadas en un búfer de ventana.